School of Engineering

Institute of Mechatronic Systems (IMS)

Project thesis System Engineering

Betaflight – Offline Controller Tuning

Author(s)		Yuri Bianchi Janick Dort Dario Jurietti
Main supervisor		Michael Ernst Peter
Secondary supervisor		Ruprecht Altenburger
Date		19.12.2025

Test für das zitieren [**zotero-item-80**]

1 Abstract

This project focuses on the development of a tool for offline tuning of gyro controllers for FPV racing drones based on the open-source firmware Betaflight. The quality of the gyro control loop has a direct influence on flight stability, the suppression of disturbances such as prop wash, and the agility of the drone. Until now, rate controller tuning has mostly been performed through time-consuming trial and error.

The aim of the project is to develop a modular, open-source solution for offline tuning based on an existing proof of concept. The code will be implemented in MATLAB, accompanied by a lightweight Python version, and supplemented with detailed Markdown documentation and examples on GitHub. The project aims to provide the community with a simple way to efficiently optimise drone parameters and thereby improve flight performance.

The results will be published in a public GitHub repository and documented in a report. In addition to the software development, the project also includes practical work with FPV drones to validate the developed methods.

2 Foreword

This document summarizes the results of our project thesis, which we worked on during the fall semester 2025 at the ZHAW School of Engineering. For us as systems engineering students, this project provided an ideal opportunity to apply advanced control theory and signal processing methods to a real, highly dynamic system. FPV drones present an interesting engineering challenge due to their highly dynamic behaviour and the need for precise control. This motivated us to investigate mathematical modelling, control engineering aspects, and offline tuning methods aimed at improving flight performance.

Our motivation was to translate the theoretical control concepts we learned over the past three years to a real, highly dynamic system. We also aimed to develop a tool that not only meets engineering standards but also provides real value to the FPV community by replacing subjective tuning with objective data analysis.

This work was carried out at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. We would like to express our gratitude to our supervisor, Michael Peter, for his guidance and support throughout this project. His expertise, constructive feedback, and willingness to take the time to assist us in developing complex algorithms were invaluable.

Inhaltsverzeichnis

3 Introduction

First-person-view (FPV) racing drones require precise and responsive control to achieve high flight stability, minimize disturbances such as prop wash, and maintain agility during rapid manoeuvres. Proper tuning of the rate controllers is key to this performance, as it directly determines how the drone reacts to pilot inputs and external influences. Until now, tuning these controllers has mainly relied on iterative trial-and-error methods, a process that is both time-consuming and difficult to perform, especially for less experienced pilots.

Betaflight, an open-source firmware widely used in the FPV community, provides a flexible platform for configuring flight controllers. However, despite its popularity, efficient tuning remains a challenge. To address this, our project builds upon an existing proof of concept for offline tuning of rate controllers using flight data and system identification techniques. The goal is to transform this concept into a robust, modular, and open-source library that simplifies the tuning process.

The library will be implemented in ‘MATLAB and Python‘, offering tools for analyzing flight logs, estimating system dynamics, and optimizing controller parameters without requiring repeated flight tests. A documentation is created, which shows how to use the tool and will be published on GitHub to ensure accessibility for the FPV community.

Besides software development, the project involves also building an FPV drone and testing and confirm the effectiveness of the proposed methods. By providing a structured, data-driven approach to tuning, this work aims to improve flight performance and reduce the barriers to achieving optimal control settings for both hobbyists and professionals.

3.1 Initial situation

The open-source firmware Betaflight is the most widely used platform for FPV racing drones and is continuously developed by a large community. The quality of the rate control is crucial for flight performance, as it enables stability, agility, and effective suppression of disturbances such as prop wash.

Today, tuning of controller parameters is still largely performed by trial-and-error adjustments, a method that is time-consuming and often challenging for less experienced pilots. A proof of concept (developed by Michael Peter) already exists, which enables offline tuning of rate controllers using flight measurement data obtained from chirp signal excitation. However, the current implementation is based on MATLAB, which is not widely used within the FPV community and therefore limits accessibility. In addition, the proof-of-concept requires a certain level of technical knowledge to operate, which further complicates its practical use for many pilots.

3.2 Task and Objective

The aim of this project is to further develop the existing proof of concept into an accessible and practical tool for tuning rate controllers on Betaflight-based FPV drones. The focus lies on transforming the prototype from a research-oriented MATLAB implementation into a solution that can be used by the wider community with minimal technical barriers. This includes designing a clean and modular software architecture, improving usability, and integrating functions for importing, processing, and analysing flight data.

In addition, the tool will be accompanied by comprehensive Markdown documentation, including explanations of key methods, usage instructions, and practical examples to support new users. The implementation will serve as a foundation for future developments, such as extended analysis features, alternative controller designs, or deployment in other embedded flight control systems. The overarching objective is to provide a reliable, data-driven workflow that replaces subjective trial-and-error tuning and enables verifiable control performance improvements.

4 Preparation

4.1 Chirp

4.1.1 Chirp Concept

A chirp is an input signal that sweeps continuously over a defined range (e.g. from a low frequency to a high frequency). This signal excites the system on multiple frequencies in one run and thus lets you observe the frequency response and step response, the foundation of controller tuning. Mathematically, a chirp signal can be represented as a sinusoidal wave with instantaneous frequency that varies linearly or exponential with time. For more information about the mathematical background, please have a look in the document Chirp Signal [**bf_chirp**].

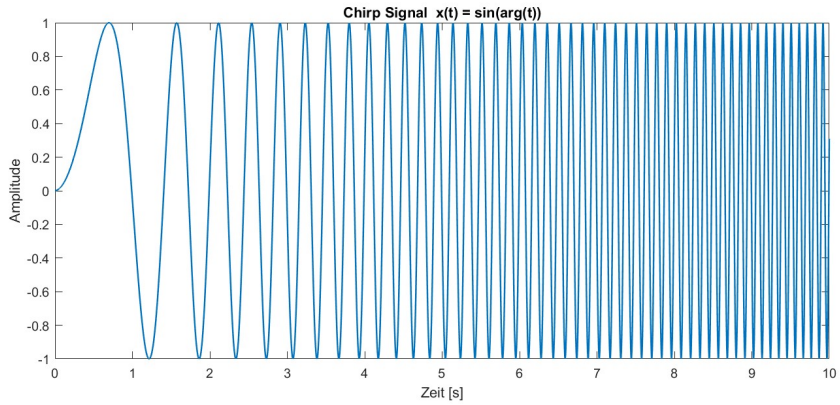


Abbildung 1: Chirp Signal example

4.1.2 Chirp on a drone

In the context of tuning a drone's flight controller, an exponential sweep is used. This is due to the dominant low-frequency dynamics, where low frequencies exhibit more predictable responses, allowing the sweep to allocate more time to higher frequencies for better resolution of system behavior. Additionally, multirotor vibration resonances are typically spaced logarithmically, making exponential sweeps more suited for efficient excitation across the frequency range.

The process involves these steps:

1. Signal Injection: The chirp signal is added to the pilot's stick input. This composite signal becomes the setpoint for the PID controller.
2. System Response: The drone attempts to follow this rapidly changing setpoint. The motors react, and the drone will oscillate at varying frequencies corresponding to the chirp signal.
3. Data Logging: While the chirp is active, high-resolution data from the gyroscope and the motor outputs are recorded using Betaflight's Blackbox feature. This data captures how the drone responds to the control inputs at different frequencies.

The recorded log file, containing the input chirp and the corresponding gyroscope output, is then used for offline analysis to determine the frequency response of the system.

4.1.3 Chirp programming

The implementation of the chirp signal in Betaflight is straightforward. You call the function chirpInit() and pass:

- a struct of type chirp_t defined in chirp.h
- starting frequency
- end frequency
- signal length
- loop-time in microseconds

This initialises the values in the data struct and:

- converts loop period from microseconds to seconds
- calculates the number of samples in the signal duration
- calculates the exponential growth factor
- precomputes the constants used to get the phase from the frequency evolution
- calls chirpReset() function

The chirpReset() function sets:

- internal counter to 0
- bool isFinished to false

and then calls the function chirpResetSignals() to set signal-related fields to 0.
Once the chirp is initialised use the chirpUpdate() function once per loop iteration.
This function checks:

- if bool isFinished is true -> return false
- else if internal counter is equal to the number of samples inside the chirps period -> set bool isFinished true, call chirpResetSignals(), return false

if none of the above is true the ...

5 Theory

5.1 Signal Processing

Signal processing applied to Betaflight Blackbox logs transforms raw flight data into usable information for in-depth analysis. These logs contain flight measurements such as stick inputs, gyro signals, PID responses and more. Although the dataset may appear complex at first, software solutions including our analysis tool and the Betaflight Blackbox Explorer make interpretation more straightforward by providing visualisations and metrics that support troubleshooting, tuning and performance evaluation.

Preparing the logged data is a central part of the workflow. Before any analysis can be performed, the raw signals are conditioned into a consistent and meaningful representation. This typically begins with filtering to suppress noise without altering the underlying dynamics. Based on the processed signals, the frequency response of the control system can be estimated to characterise how it reacts to different input spectra. Each processing step increases the reliability of the data and leads to more robust conclusions in subsequent analysis.

5.1.1 Data Import

The first step of any analysis is to load the measurement data properly. Betaflight Blackbox logs consist of two essential parts: the header and the data block. The header contains key information about the conditions under which the flight data was recorded, including loop rates, logging settings, active filter configurations and the PID controller values. Without this information, calculations cannot be performed correctly and the recorded data loses its analytical value.

Data imports are unfortunately very computationally expensive. Therefore, to save time during analysis, the logs are converted from raw CSV files into a faster '.mat' format. This way, you don't need to reprocess the large log files every time. The signal names (like gyro and motor outputs) are also automatically assigned to their correct data columns. This makes it easy to find and assign the signals later. For more details, check the Dataimport Documentation [**bf_dataimport**].

5.1.2 Handle High-Resolution Data

Betaflight provides a High Resolution Mode as an optional logging setting, in which selected signals are multiplied by a factor of ten before being written to the log. Because these values are stored as integers, this reduces decimal precision requirements and minimises rounding effects. During analysis, the affected signals must be scaled back to their original units; otherwise, frequency plots, controller tuning results or general interpretations will be incorrect. Important signals such as gyro data and control inputs are affected by this step. To get more information, check the Unscale high Resolution Gain documentation [**bf_unscale_highres**].

5.1.3 Time Conversion and Sampling Rate Verification

In a Betaflight log, time is stored as an increasing microsecond counter rather than as regular seconds. Because of hardware limits or system load, the spacing between time stamps isn't always the same. This step converts the time information into a uniform time base, ensuring that all samples can be compared consistently and interpreted in their correct temporal context.

By checking the time differences between samples, you can detect logging issues such as dropped frames or small variations in the actual sampling rate. Correct handling of the time information is important for all further analysis steps, for example frequency calculations or controller tuning. For a more detailed explanation, see the Convert and evolution Time documentation. If you are interested in tuning your drone, you can also refer to the Evolution Time Flight description.

5.1.4 Application of Rotational Filtering (Apply Rotfiltfilt)

Filtering is a fundamental step in preparing the data for analysis, as it removes unwanted noise while preserving the original characteristics of the signals. The function `Apply Rotfiltfilt` operation applies a zero-phase filter, ensuring that the data's phase relationships remain intact while suppressing selected frequency components such as low-frequency drifts or high-frequency noise.

This step applies a forward-backward filtering technique, which processes the data in both forward and reverse directions and effectively cancels out phase distortions. This method is particularly useful when working with chirp signals, as it enables precise filtering without introducing phase shifts that could alter the signal's characteristics.

Proper application of this technique ensures that the data is clean and reliable for subsequent analysis steps. For more insights on how this method works, refer to the full description in the Apply Rotfiltfilt documentation.

5.1.5 Estimation of Frequency Response

The `Estimate Frequency Response` function uses advanced techniques to measure and analyze the relationship between a system's input and output signals in the frequency domain. This makes it possible to determine how different frequencies are amplified or attenuated by the system, providing valuable information about its dynamic behavior.

This technique is based on the Welch Method, which divides the signal into overlapping segments. By applying a Hann window to each segment and performing spectral averaging, the influence of noise is reduced and a smoother estimate of the frequency response is obtained. The result is a single-sided, amplitude-calibrated response curve that shows how individual frequencies are amplified or attenuated, as well as how phase shifts occur across the spectrum.

The resulting data is not only accurate but also robust, making it suitable for applications like control system tuning or stability analysis. For a deeper dive into the theoretical principles and implementation steps, consult the detailed Estimate Frequency Response documentation.

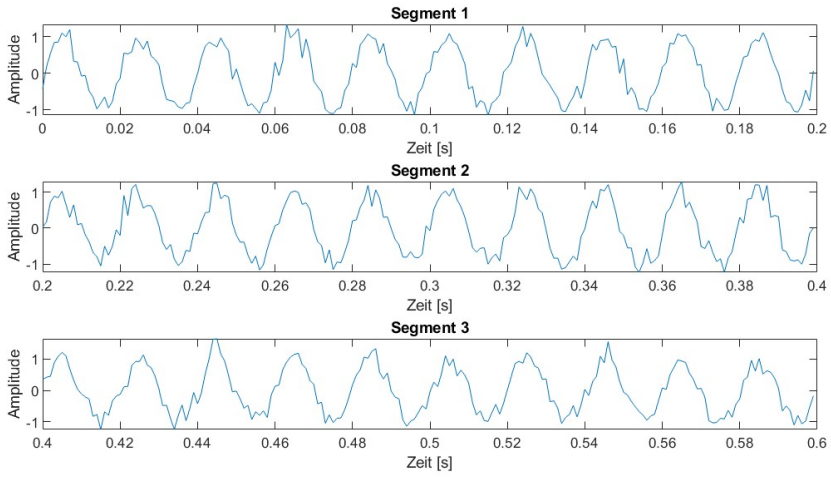


Abbildung 2: Segmentations of Data

5.2 Control System

The control system determines the dynamic response of a drone and is therefore central to flight stability and precise maneuvering. In this section, we analyse the individual components of the controller, examine their influence on the system behaviour and outline the methods used for tuning in order to improve performance and responsiveness.

5.2.1 Transfer Function Estimation

To analyse how the system reacts to control inputs, the transfer function of the plant is estimated directly from closed-loop flight data. This makes it possible to characterise the system dynamics without opening the control loop and to determine how input signals are transformed into outputs. The resulting model forms the basis for matching the controller parameters to the real system. More details can be found in the Estimate Transfer Function documentation.

5.2.2 Closed Loop Calculation

The interaction between the controller and the estimated transfer function of the plant determines the stability and robustness of the closed-loop system. By calculating the closed-loop transfer functions for both inner and outer loops, sensitivity, disturbance rejection, and overall system stability can be evaluated. These calculations provide a clear view of the controller dynamics and their impact on the system response. For details on the implementation, see the Calculate Closed Loop documentation.

5.2.3 Filters and Controllers

Filters and controllers form the foundation of the control loop. Filters remove unwanted noise from sensor data or motor outputs, while controllers ensure that the flight system remains stable and tracks the desired setpoints accurately. The system makes use of various low-pass, notch, and dynamic filters that are designed to optimize the control process. The controllers, such as the PI

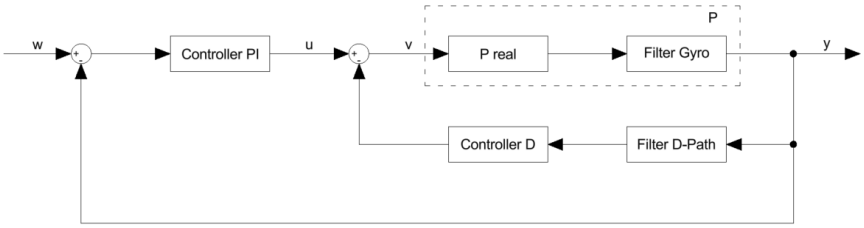


Abbildung 3: Gyro Control Loop

and D components, are then tuned to achieve the desired balance between reactivity and stability. For further insights, see the Filters and Controllers documentation.

5.2.4 Compensation of the I-Term

The integral term in a PID controller addresses long-term errors, ensuring steady corrections over time. However, it can become problematic during quick stick movements, where it may overcorrect and destabilize the system. To avoid this, compensation methods adjust the influence of the I-term based on the situation, suppressing it during rapid changes while allowing it to integrate errors during slower movements. This approach helps maintain both precision and responsiveness. Have a look at Compensate I-Term documentation to see how we added compensation to the calculation.

5.3 Evaluation

Evaluation is an essential part of understanding and improving the behaviour of the control system. This section outlines the key analytical steps, such as analysing time-domain responses and interpreting frequency-domain data to evaluate the system dynamics. Detailed information on how to interpret the generated figures can be found in the Descriptions Figures.

5.3.1 Step Response

The step response provides a direct view of how the system reacts to sudden changes in the input signal. By transforming the closed-loop transfer function from the frequency domain into the time domain, the dynamic behaviour of the control loop can be evaluated. Key performance indicators such as rise time, overshoot, settling time, and steady-state error can be determined to assess system stability and responsiveness. The calculations use inverse Fourier transforms of frequency response data to reconstruct the system's step response curve. This curve can be used to identify underdamped or insufficiently tuned dynamics. More details can be found in the Step Response documentation.

5.3.2 Spectra Analysis

Spectral analysis examines how the system responds to different input frequencies and provides valuable insight into its dynamic characteristics. The method separates complex signals into their frequency components using algorithms such as Welch's method. Overlapping signal segments, combined with windowing techniques like the Hann window, yield smoother and statistically more reliable estimates of power spectra. This frequency-domain perspective highlights resonances, noise characteristics, and the effectiveness of applied filters. The steps of this process are described in detail in the Spectra Analysis documentation.

5.3.3 Spectrum Analysis

Spectrum analysis builds on spectral analysis by providing a multi-dimensional view of the signal's frequency content over time or, in this case, thrust. By grouping FFT segments into bins along an additional axis, such as throttle or angle, the power spectrum can be mapped as a function of both frequency and the selected variable. This approach is particularly useful in applications that require an understanding of how system dynamics change during flight. Relevant methodologies and results are described in the Spectrum Analysis section.

5.3.4 Gang of Four

The Gang of Four plots provide a compact overview of the closed-loop behaviour of the control system. They show how the tuned controller influences stability, noise handling, and control authority. The Bode Diagram illustrates the overall frequency response of the closed loop. The Sensitivity curve indicates how well disturbances are rejected at different frequencies, while the Controller Effort shows the control action required from the PID loop. Finally, the Compliance plot reflects how external disturbances pass through the system.

These diagrams form an essential basis for controller tuning, as they allow a direct comparison between the measured system behaviour and the simulated response using the estimated transfer

functions. For a detailed explanation of the control relationships, refer to the Calculate Closed Loop documentation. Practical tuning guidance can be found in the section Gang of Four Figures.

6 Challenges and Methodological Approach

6.1 Simulations

At the beginning of our thesis, we started creating simulations. We used these simulations to deepen our understanding of the existing code and the methods it employed. Although the simulations were time-intensive, they proved crucial for our work. They formed the basis for all subsequent analysis steps and helped us understand the existing functions before applying them further.

6.1.1 Simulation of Betaflight Controller Order

To analyse and better understand the Betaflight control system, we recreated the complete control structure in MATLAB. The corresponding script can be found in the attachments. In this simulation, we attempted to reproduce the step response of the recorded data by using the same PID parameters and filter settings as in the real system. The different filters were created according to the documentation Filter and Controller. We implemented the filters both in continuous and discrete form, allowing us to compare the two approaches. The PI and D controllers were built in the same way. For the plant transfer function, we used a low pass filter combined with dead time, based on the approaches described in the documentation Calculation Closed Loop. In addition to the MATLAB code, we created a Simulink model to test the step response of our Betaflight control system simulation. The simulated response closely matched the step response of the real drone, confirming that we had correctly modelled the controller order.

6.1.2 Simulation PID Controller

To deepen our understanding of the different controller components, we created an additional MATLAB script. This script allowed us to combine various controller types with simple plant models. The user can switch between P, I, PI, PD and PID controllers, as well as between basic plant types. This enabled us to observe how each controller component influences the overall system behaviour. The script calculates the open-loop and closed-loop transfer functions for the chosen settings and generates Bode plots, Nyquist diagrams, and step responses. This helped us understand and visualise the effect of each component, significantly improving our overall understanding of controller behaviour.

6.1.3 Simulation Frequency Response

To analyse the frequency response of a system based on input and output data, we created a dedicated simulation. In this simulation, we generated a chirp signal with added noise to replicate realistic measurement conditions. Using these signals, we implemented different methods to estimate the system's transfer function. First, we calculated a basic frequency response using the plain **F**ast **F**ourier **T**ransformation (FFT), which served as a benchmark. After that, we applied various signal processing methods. We began with the Welch method to improve the estimation by averaging over overlapping segments. The implementation of the Welch **F**requency **R**esponse **F**unction (FRF) follows the theory described in the documentation Estimate Frequency Response. This allowed us to compare the plain FFT approach with the more robust Welch estimator. In the next step, we implemented a rotated-signal approach, where the chirp is demodulated to baseband before filtering. For this part, we used the zero-phase filtering method apply_rotfiltfilt, as described in the documentation Apply Rotfiltfilt. We then compared the original Bode plot to the processed one

to evaluate how the rotation technique improves noise robustness. Overall, this script helped us understand how noise affects the estimated transfer function and how signal processing techniques can increase robustness.

6.1.4 Simulation Spectra

We developed a simulation to examine the different components used to create a meaningful spectrum, based on the system used in the existing code. In this script, we generated a noisy sinusoidal input signal from which we created different spectra. In one case, we applied a window function, while in the other we used no window at all. By splitting the signal into several overlapping segments, we observed how averaging across segments reduces noise and stabilises the resulting spectrum. The implementation is described in detail in the document Spectra Analysis. This exercise helped us understand how windowing, overlap, and segment length affect spectral estimation, as well as how these methods can increase robustness and reduce spectral leakage.

6.1.5 Simulation Spectrogram

To analyse how the frequency content of the excitation signal changes with both time and thrust, we developed an additional MATLAB simulation. In this script, we generated a noisy sinusoidal input signal and a slowly varying thrust signal. The input signal was then divided into segments and windowed with a Hann window. Each segment was transformed using the FFT to obtain a time-frequency representation. First, we arranged the segment spectra along the time axis, and in a second step, along the thrust levels. This resulted in two spectrograms: one showing amplitude over frequency and time, and the other showing amplitude over frequency and thrust. Through this visualisation, we could identify the dominant frequency components as a function of thrust. The underlying method is described in detail in the document Spectrogram Analysis.

6.2 Building an FPV drone

Besides the understanding of the code and further developing the proof of concept, building a drone and set it up correctly bridges the gap between the digital domain, where algorithms and simulations are developed and the physical reality of drone operation. By getting in touch with the hardware, it is easier for us to understand what practical challenges exist for pilots and the community.

Abbildung 4: aosmini 3-inch build

Therefore a three inch FPV drone was built from the ground up. This work included:

- creating schemes to see which connections were required
- soldering all the components as the motors, receiver, video system, logger and gps module to the flight controller
- attaching everything to the frame, so that nothing generates unnecessary oscillations during flight
- double check all soldering connections and screws
- setting everything up in betaflight
- testfly the drone

The flightcontroller just fitted into this frame. The GPS mount had to be trimmed off a little bit, that we could mount the all in one stack. Also the video system was mounted super close to the stack. Due to a connector between these two parts, we had to lift the videosystem up with some spacers. This resulted in not enough space on top of the videosystem for the screws, which hold the component. So the videosystem had to be mounted with just two screws instead of four. In the

first testflights, we couldn't see any too hard vibrations from this, so it's probably mounted good enough.

The logger is mounted on the bottom of the drone... -> we designed some stands

The biggest challenge was to achieve good soldering connections, ...

6.3 Tuning an FPV drone

As part of understanding the workflow we tuned an FPV drone by ourselves. The goal was to adjust the PID parameters so that we achieve a fast rising step response without overshoot, a lower compliance than before and a rounded shaped Sensitivity. Also to mention is, that the axes roll and pitch should be tuned as similar to each other as possible. This is because you want the drone to maintain a consistent and predictable response, so roll and pitch behave similarly for smoother control and a balanced flight experience.

Initially, we deliberately applied a poor tuning configuration to observe its effects. During the first field test with this setup, it quickly became apparent that the tuning was excessively poor. The drone began oscillating aggressively and climbed rapidly. As the drone was out of control, the kill switch had to be activated, resulting in a crash that broke one of the four arms. Fortunately, the damage was minor and repaired quickly.

After this incident, we decided not to repeat flights with intentionally bad tuning. Instead, we continued using the default Betaflight parameters as a baseline. With these the drone flies quite good. However, during demanding manoeuvres such as flips or propwash manoeuvres, it becomes evident that there is room for improvement in terms of stability and responsiveness.

The tuning process was carried out on a larger 6-inch FPV drone provided by one of the team members. The drone was freshly flashed with Betaflight v12, meaning all filters and settings were at factory defaults. Achieving an optimal tune required several attempts due to challenges with filter adjustments and occasional logging issues. Nevertheless, the drone was successfully tuned, and the improvement was noticeable. During propwash manoeuvres, the drone remained significantly more stable, and flips were executed with greater precision at the stop.

Our tuning objective was to achieve a fast step response without overshoot, while maintaining good compliance and sensitivity. It is important to note that there is no universally "perfect" tune. Settings depend heavily on pilot preferences. Racing pilots, for example, may want more responsiveness for competitive performance, whereas freestyle pilots want smoothness and flight feel.

Since none of our team members are professional pilots, we focused on achieving robustness and a step response without overshoot. This tuning approach is probably somewhere between what a racing pilot and a freestyle pilot would prefer. Robustness in this context is important for practical scenarios, such as when a propeller is slightly bent, the battery sits slightly off center or when an additional component like a camera is mounted on the drone.

6.4 Decoding Betaflight Blackbox Logs

6.4.1 Purpose

Currently to obtain a useable Blackbox log from Betaflight, pilots rely on the Betaflight Explorer. Wich is a reliable tool, but lacks comfort if the tuning tool is used iteratively since you have to visit the Explorer every time you want to download a new log. To improve this workflow, we tried to make use of the existing open-source blackbox_decode tool.

6.4.2 blackbox_decode tool

The blackbox_decode tool is a C-based command-line utility designed to convert Betaflight Blackbox log files from their proprietary binary format into more accessible formats such as CSV. It offers similar core functionality to the Betaflight Blackbox Explorer but is intended for users who prefer command-line tools or need to automate log processing tasks.

However, integrating blackbox_decode into our project presented several challenges. The most obvious issue is the absence of header information by default. While this can be addressed by using the –save-headers flag, the format of these headers differs from that used by the Betaflight Explorer. This discrepancy is manageable, as we could adapt our data import functions to handle both formats.

A more significant problem is the presence of numerical differences in the decoded data. When comparing logs decoded with blackbox_decode to those processed with Betaflight Explorer, the resulting datasets differ structurally and semantically in several critical ways. Specifically, the blackbox_decode stores the vehicle attitude as Euler angles in degrees, whereas the Explorer provides quaternion-like components. With priority set on other tasks, we decided to set aside the integration of blackbox_decode for now.

6.5 Code Structure and programming style

From the start of the project, we recognized that a clear and maintainable code structure would be essential. The initial proof-of-concept code already contained a rough separation into topics such as data I/O and step response, but everything was packed into a single file, making it difficult to navigate. Our first step was to comment out the proof-of-concept code and reorganize it into thematic sections. This was the foundation for a more organized layout and we came to an initial directory structure:

```
bf_controller_tuning/
├── main.m
├── data_io/
│   ├── load_data.m
│   ├── extract_header_information.m
│   └── preprocess_data.m
├── flight_parameters/
│   ├── get_pid_parameters.m
│   ├── scale_pid_parameters.m
│   └── print_parameters.m
├── spectrogram/
│   ├── calculate_spectrogram.m
│   ├── plot_spectrogram.m
│   └── show_spectrogram.m
├── spectra/
│   ├── estimate_spectra.m
│   ├── plot_spectra.m
│   └── show_spectra.m
├── frequency_response/
│   ├── estimate_frequency_response.m
│   ├── plot_bode.m
│   └── show_frequency_response.m
├── controller_analysis/
│   ├── calculate_closed_loop.m
│   ├── calculate_step_response.m
│   └── show_controller_analysis.m
└── utils/
    ├── apply_rotfiltfilt.m
    ├── downsample_frd.m
    └── get_my_colors.m
```

This draft was never intended to be final. Rather, it served as an initial attempt to bring clarity and separation between different functional areas.

6.5.1 OOP vs. Functional Approach

At this stage, we discussed whether to implement the entire project using object-oriented programming (OOP) or stick to a modular, function-based approach. Initially, we considered OOP as a challenge, since most of us were not very experienced with it. We started writing some components

as classes, but quickly realized that only one team member was really comfortable with OOP, while the others struggled. This made us reconsider. Our conclusion was to use OOP selectively—only for parts that benefit from reuse and encapsulation, such as data loading and plotting, while keeping the rest function-based for simplicity. This hybrid approach allowed us to maintain flexibility without overcomplicating the design.

After several attempts, we agreed on the following principles:

- A main script (main.m) for user input, figure selection, filter settings, and PID tuning
- A core class to handle all calculations (essentially the logic from the proof of concept, but without plotting)
- A plot utilities class to centralize all plotting functions for consistency

Existing library functions from the proof of concept were reused without modification. The thematic grouping included:

```
bf_controller_tuning/
├── class/
│   ├── main_class.m
│   └── plot_utils.m
├── lib/
│   ├── apply_rotfiltfilt.m
│   ├── calculate_closed_loop.m
│   ├── calculate_controllers.m
│   ├── calculate_step_response_from_frd.m
│   ├── calculate_transfer_functions.m
│   ├── downsample_frd.m
│   ├── estimate_frequency_response.m
│   ├── estimate_spectra.m
│   ├── estimate_spectrogram.m
│   ├── expand_multiple_figure_nr.m
│   ├── extract_header_information.m
│   ├── get_chirp_signals.m
│   ├── get_fcut_from_D_and_fcenter.m
│   ├── get_fcut_from_exp.m
│   ├── get_filter.m
│   ├── get_ind_eval.m
│   ├── get_my_colors.m
│   ├── get_notch_Q.m
│   ├── get_pid_scale.m
│   └── get_switch_case_text_from_para.m
├── logs/
└── main.m
```

This structure worked well and kept the main script simple for the user. However, after a review meeting, our instructor suggested further modularization. The reason: we had created a “god class,” which limited flexibility. For example, if someone wanted to use the tool only for plotting spectrograms without any drone-specific logic, the current design made that difficult. Also the

program was slow. All calculations were made, every time the user changed something. We wanted to improve the speed in only doing these calculations again, which are relevant on that specific user input change.

6.5.2 Final Structure Concept

Out of this, we decided to split up this god class into three classes. The `flight_analyzer` class handles frequency domain analysis and vibration characterization. Its core responsibility is performing spectral analysis on flight log data to identify resonance frequencies and assess filter effectiveness. It computes power spectral density of gyro signals (both filtered and unfiltered) and generates spectrograms showing how vibration frequencies vary with throttle levels across all three axes (roll, pitch, yaw). This analysis is essential for identifying noise sources, evaluating filtering strategies, and optimizing drone stability during tuning. The `flight_data` class is responsible for loading, parsing, and managing flight log data. It provides methods to extract relevant information from Blackbox logs, such as gyro and motor data, and prepares this data for analysis. The `gyro_ctrl_tuning` class focuses on tuning the PID controllers for the drone's flight. It uses the results from the frequency response and spectral analysis to adjust the PID parameters, aiming to optimize the drone's flight characteristics. The `plot_utils` class remains responsible for visualization and graphical representation of all analysis results. Unlike the previous approach, this class is now more flexible and reusable. It can be used independently to generate plots from pre-calculated data, making it suitable for users who only want to visualize spectrograms, frequency responses, or step responses without running the full analysis pipeline. This modularity allows the plotting functionality to be easily adapted for different visualization needs and integrated into other workflows.

This approach still holds thematic sections, but is more accessible to use only specific parts of the tool.

```
bf_controller_tuning/
├── class/
│   ├── flight_analyzer.m
│   ├── flight_data.m
│   ├── gyro_ctrl_tuning.m
│   └── plot_utils.m
└── lib/
    ├── apply_rotfiltfilt.m
    ├── calculate_closed_loop.m
    ├── calculate_controllers.m
    ├── calculate_step_response_from_frd.m
    ├── calculate_transfer_functions.m
    ├── downsample_frd.m
    ├── estimate_frequency_response.m
    ├── estimate_spectra.m
    ├── estimate_spectrogram.m
    ├── expand_multiple_figure_nr.m
    ├── extract_header_information.m
    ├── get_chirp_signals.m
    ├── get_fcut_from_D_and_fcenter.m
    ├── get_fcut_from_exp.m
    └── get_filter.m
```

```
    ├── get_ind_eval.m
    ├── get_my_colors.m
    ├── get_notch_Q.m
    ├── get_pid_scale.m
    └── get_switch_case_text_from_para.m
├── logs/
└── main.m
```

6.6 Conversion from MATLAB to Python

To make the project more accessible to the FPV community, we decided to migrate the existing MATLAB codebase to Python. The first step was to select suitable Python libraries that offer similar functionality to MATLAB, while maintaining code efficiency and readability. After evaluating several options, we chose the following libraries:

- **NumPy**: For numerical operations and array manipulations, analogous to MATLAB's matrix operations.
- **SciPy**: For scientific computing tasks, including signal processing functions comparable to MATLAB's capabilities.
- **Matplotlib**: For plotting and visualization, providing features similar to MATLAB's plotting tools.
- **Pandas**: For data manipulation and analysis, useful for handling tabular data as in MATLAB tables.
- **Control Systems Library (control)**: For control system analysis and design, offering functions equivalent to MATLAB's Control System Toolbox.

We selected Python version 3.12, as it is well-supported and compatible with the required libraries. To validate these choices, we developed a prototype script that implements the core functionalities needed for the Drone Tuning Tool. The script loads flight log data from CSV files, parses and caches the data efficiently, and extracts relevant signals such as setpoints and gyro measurements. NumPy and Pandas are used for numerical and tabular data handling, while SciPy provides signal processing utilities for windowing and frequency analysis. The Control Systems Library is employed to estimate the system's frequency response and step response from flight data, replicating MATLAB's identification routines. Matplotlib is used for visualizing raw signals, chirp responses, and system characteristics. The modular structure, including custom data classes and caching, ensures fast and reproducible analysis. This approach demonstrates that Python and its scientific libraries are suitable replacements for MATLAB in the drone tuning workflow. While Pandas is not strictly necessary for the prototype, it simplifies data handling and improves code readability. However, to minimize dependencies and ensure cross-platform compatibility, we may consider removing Pandas in the final implementation. The flexibility of Python also allows for easier integration with other tools and automation of data processing tasks, which is beneficial for large-scale log analysis.

6.7 Python in MATLAB

In our initial porting attempt, we utilized MATLAB's built-in functionality to execute Python code directly from within the MATLAB environment. While this approach allowed us to access Python libraries and scripts, it introduced some challenges. One major issue was the frequent need for unit conversions between MATLAB and Python data types, which often led to subtle bugs and inconsistencies. Additionally, we encountered differences in the implementation of certain algorithms, such as varying scaling conventions in the state-space representation of transfer functions. These discrepancies complicated the integration process and made it difficult to ensure consistent results across both platforms. Managing library dependencies and Python environments within MATLAB also proved cumbersome, especially when working across different operating systems or MATLAB versions. As a result, this method proved unreliable for our project's requirements. We ultimately

decided to pursue a native Python implementation to avoid these integration issues and streamline the development workflow.

6.8 Transferring the MATLAB Library to Python

The core functionality of the Drone Tuning Tool resides in its library functions. To ensure that the Python implementation is both reliable and consistent with the original MATLAB code, we systematically transferred each library function from MATLAB to Python. For functions with simple numerical outputs, we used `np.testing.assert_allclose` with appropriate absolute and relative tolerances, saving the function inputs and outputs from MATLAB and comparing them with the Python implementation. For more complex functions, such as those involving state-space outputs, we developed dedicated test scripts to compare the behavior of the MATLAB and Python versions across a range of scenarios. This rigorous testing approach helped us validate the correctness and consistency of the ported library functions. Automated testing and comparison between MATLAB and Python outputs were essential to ensure that the new implementation maintained the expected performance and accuracy. This systematic transfer and validation process significantly reduced the risk of introducing errors during migration and increased confidence in the reliability of the Python-based tool.

7 Results

Throughout the project, we achieved a range of practical and technical results. The most important outcome was gaining a deep understanding of the Betaflight control system, including how its filter structures and controller components interact to stabilise the gyro control loop. In addition, we developed a solid foundation in the application of essential signal-processing methods and learned how to use them correctly for frequency-domain analysis, step-response estimation and data preparation. This newly acquired knowledge will be crucial for the further development of the tool and forms an important basis for the work that will follow in the bachelor's thesis. The simulations carried out during the project helped us link theoretical concepts to the real dynamic behaviour of an FPV drone and validate our understanding of the control loops.

Alongside the theoretical work, we also built and configured an FPV drone from scr atch and performed flight tests and tuning. This hands-on experience provided valuable insight into the practical challenges of drone construction, setup, flight and tuning. It helped us better understand the real-world implications of our analysis, the needs of the FPV community, and ultimately strengthened the connection between theory and practice.

On the software side, we restructured the existing prototype into a modular codebase, created comprehensive Markdown documentation on GitHub, and began transferring key functions from MATLAB to Python to enable future extensions. We also gained experience in object-oriented programming and code organisation for larger projects. In addition, we made significant progress in migrating core functions from MATLAB to Python, such as closed-loop calculations and transfer-function estimation. Although the full transfer is still ongoing, the results achieved so far provide a strong foundation for further development and refinement during the bachelor's thesis.

A particularly time-consuming part of the project was the development of an automatic extraction method for flight data directly from Betaflight Blackbox logs. Despite testing different approaches, we were not able to complete a reliable solution within the given time frame. However, the insights gained during this process form a valuable basis for further investigations, and we plan to revisit this topic as part of the bachelor's thesis.

In summary, the project demonstrates that offline analysis and tuning of Betaflight-based FPV drones provides a robust, objective and data-driven approach to optimising flight performance. The implemented methods enable the reconstruction of control-loop behaviour from flight measurements, allowing for a detailed understanding of system dynamics and the impact of controller settings. The work carried out so far lays a solid foundation for the bachelor's thesis, during which the tool will be extended with additional tuning capabilities, fully migrated to Python, and validated through further flight tests.

8 Discussion and next steps

8.1 Discussion

Overall, the work in the last 14 weeks mainly served as a foundation for the future work at the project during our Bachlor thesis rather than a finished end product. We were able to lay the groundwork for further developments. For example, we newly structured the code, which allows us to make addtions and upgrades in the code faster and clearer. But more importantly, we were able to lern much about the behaviour of the Betaflight controll system and how we have to face challenges in it. Dario Jurietti and Janick Dort also lerned how a drone flys and how to build an FPV drone. This experience helped them to better relate the abstract analysis results and behaviour in the air.

Additionally to this, we deepened our understanding how advanced signal-processing methode and more complex controll loops working. We achived this through a series of simulations which made it us easier to connect the theorie to the measuered data of real flights. Also, we improved our skills in object oriented programming and how to structer a big code. We also were able to get our first experiene in using control-engineering in Python, even though the Python implementation is not finished yet. All of this will help us to develope new features quickly and with high quality in our Bachlor thesis.

8.2 Next Steps

This project is being continued and further developed as part of a bachelor's thesis. In this context, we plan to extend the existing codebase with new features such as offline tuning for position and altitude hold. These additions will provide deeper insight into the behaviour of the corresponding control loops and allow their optimisation.

Furthermore, a full transfer of the implementation from MATLAB to Python is planned. This will make the tool more accessible to the FPV community, as Python is open source, widely used and free of charge. Our long-term goal is to enable seamless integration of the tool into existing Betaflight utilities, so that pilots can use it for offline tuning of their drones.

Finally, we intend to validate the analysis results through additional flight tests. By comparing the outcomes of the offline analysis with real flight behaviour, we aim to ensure that the methods are accurate and reliable. This will help us further refine the tool and increase its practical value for the FPV community.

8.3 Conclusion

Throughout the project, we gained a deep understanding of the Betaflight control system, learned how its filter and controller structures interact, and applied advanced signal-processing techniques for frequency-domain analysis, step-response evaluation and flight-data preparation. Simulations allowed us to connect theoretical models with observed flight behaviour, while building and test-flying an FPV drone helped us translate analytical findings into practical insights. On the software side, we restructured the existing prototype into a modular codebase and began transferring key functions from MATLAB to Python, enabling future extensions and wider accessibility. Although the fully automated extraction of flight data from Blackbox logs could not be completed, the knowledge gained provides an important basis for further investigation. Overall, the results show that offline

analysis and tuning of Betaflight FPV drones offers a robust, objective, and data-driven approach to optimize flight performance. The implemented methods enable reconstruction of control-loop dynamics from flight measurements, allowing detailed understanding of system behaviour and the impact of different controller settings. The work carried out so far lays a solid foundation for further development and refinement of the tool in our bachelor's thesis, where we will add new tuning capabilities, complete the Python migration, and validate through additional flight tests.

References

8.0.1 Overview of Generative AI Systems

OpenAI (2025) – ChatGPT 5.1

- Available online: https://chat.openai.com/
- Functions:
  - Generation of text suggestions
  - Grammar and language correction
  - Support in writing LaTeX and MATLAB code

DeepL (2025) – DeepL Translator (Free Version)

- Available online: https://www.deepl.com/translator
- Functions:
  - Translation of text

Microsoft (2025) – Copilot AI Assistant

- Available online: https://copilot.microsoft.com/
- Functions:
  - Generation of text and summaries
  - Integration into Microsoft Office tools such as Word and PowerPoint
  - Assistance in software development (e.g., code suggestions)

Anthropic (2025) – Claude AI

- Available online: https://claude.ai/
- Functions:
  - Text generation and editing
  - Support for research tasks (e.g., summarisation and structuring)
  - Assistance in programming-related workflows

8.1 List of Figures

9 Appendix

9.1 Project Management

9.1.1 Project Assignment

aufgabenstellung.md 2025-09-15

Projektarbeit

Allgemein

Titel

Betaflight - Offline Controller Tuning

Beschreibung

Betaflight ist ein Open-Source Projekt, das sich auf die Regelung von FPV Quadcopter spezialisiert hat, dies insbesondere im Bereich von Race-Dronen. Die Firmware wird von einer grossen Community weiterentwickelt und ist auf vielen verschiedenen Hardware-Plattformen verfügbar. Betaflight ist die am häufigsten verwendete Firmware für FPV-Drohnen und wird von vielen Hobbyisten und Profis genutzt.

Für die Flugperformace ist die Qualität der Raten-Regelung von entscheidender Bedeutung. Die Regler sind dafür verantwortlich, dass die Drohne stabil fliegt, Störungen wie Wind und Strömungsabrisse (Prop-Wash) unterdrückt und möglichst agil auf Steuerbefehle reagiert. Die Regler und Filter müssen daher sorgfältig getunt und abgestimmt werden, um eine optimale Flugperformance zu erzielen. FPV Piloten fliegen üblicherweise im ACRO mode, sprich das System ist Ratengeregelt und die Stickinputs des Piloten werden als Sollwerte auf den Ratenregler gegeben. Das Tuning dieser Regler geschieht in der Community im Allgemeinen durch Trial and Error. Das bedeutet, dass die Piloten die Reglerparameter anpassen und dann testen. Dies kann jedoch zeitaufwendig und frustrierend sein, da es oft schwierig ist, insbesondere für Leien, die richtigen Parameter zu finden.

Bereits bestehend ist ein Proof of Concept, welches es ermöglicht die Raten-Regler offline basierend auf einem Messdaten-Modell der Drone zu tunen (basierend auf Chirp-Signal Experiment während dem Flug). Das Projekt soll auf diesem Proof of Concept aufbauen und die Methode des Offline Tunings für die Raten-Regler (Gyro) weiterentwickeln. Ziel ist es, eine vollständige, einfach erweiterbare und sauber implementierte Open-Source Libary zu entwickeln. Dies sowohl in MATLAB als auch in Python. Ausserdem soll für die Python Implementation eine ausführliche Dokumentation mit Beispielen für Anwender erstellt werden (Github Markdown Dokumentation). Das Ziel ist es, dass die Libary von der Community genutzt werden kann. Dies soll dazu beitragen, die Flugperformance der Drohnen zu verbessern und den Piloten eine einfachere Möglichkeit zu bieten, ihre Drohnen zu tunen.

Studenten

Bianchi Yuri (biancyur)
Dort Janick (dortjan1)
Jurietti Dario (juriedar)

Betreuer

Hauptbetreuer: Michael Peter (pmic)
Nebenbetreuer: Prof. Dr. Ruprecht Altenburger (altb)

Weitere Personen

Camille Huber (hurc)

Voraussetzungen

- GRT, RT1
- Zumindest eine Person ist im Besitz des Fernpiloten-Zeugnis A1/A3 (A2 optional jedoch empfohlen)
- Genügend hohe Haftpflichtversicherung, muss von den Studierenden selbst organisiert werden
- Erfahrung mit FPV Dronen (Bauen, Reparieren, Konfigurieren, Fliegen)
- MATLAB, Python

Vereinbarung

Die Vereinbarung wird zwischen dem Betreuer Michael Peter (pmic) und den Studierenden Yuri Bianchi (biancyur), Janick Dort (dortjan1) und Dario Jurietti (juriedar).

Da es sich um eine dreier PA handelt wird das Projekt in drei separat bewertbare Teile aufteilt:

- MATLAB Implementation
- Python Implementation
- Markdown Dokumentation mit Beispielen der Matlab und Python Implementation

Zeitplan

Siehe: https://intra.zhaw.ch/departemente/school-of-engineering/bachelorstudium/projekt-und-bachelorarbeiten

Projektarbeit HS 25	
Ab 15.09.2025	Ausgabe der Aufgabenstellung durch Dozierende an Studierende
Bis 19.12.2025 bis 18:00 Uhr	Abgabe der Projektarbeit in Complesis
03.02.2026	Prov. Notenfreischaltung für Studierende ab 18:00 Uhr
27.02.2026	Def. Notenfreischaltung für Studierende ab 18:00 Uhr

Arbeitsplatz

TE 617

Hardware

- Hardware kann von Michael Peter und IMS bezogen werden
  - 2 x 3.5 Zoll Dronen
  - 2 x 5.0 Zoll Dronen
- Es können auch noch zwei weitere Dronen (1 x 3.5 Zoll, 1 x 5.0 Zoll) gebaut werden

Ergebnisdarstellung

Ein Bericht im Umfang von 20-30 Seiten dokumentiert den Entwicklungsweg geeignet. Die Software wird in einem öffentlichen GitHub Repository abgelegt. Des weiteren soll eine Dokumentation mit Beispielen in Markdown erstellt werden. Die technische Dokumentation ist ebenso in Markdown zu verfassen.

Fachgebiet

Primäres Fachgebiet

Regelungstechnik und Advanced Control

Weitere Fachgebiete

- Datenvisualisierung
- Digitale Signalverarbeitung
- Software Engineering

Verschiedenes

Studiengang

Systemtechnik

Industriepartner

Keiner

Institut / Zentren

Institut für Mechatronische Systeme (IMS)

Interne Partner

Keine

Informationslink

https://github.com/pichim/bf_controller_tuning

Bemerkungen

- Grundsätzlich ist die Idee die PA aufbauend in einer BA zu erweitern

Sprache

Deutsch / Englisch

Studenten Detail

- Yuri Bianchi (biancyur) - Automatiker - Leader, Pilot, Regelungstechnik & Signalverarbeitung
- Dario Jurietti (juriedar) - Biolaborant - Datenverarbeitung, Software
- Janick Dort (dortjan1) - Elektroplaner - Datenverarbeitung, Pilot, Regelungstechnik & Signalverarbeitung

Die Studenten haben schon PM 1-4 zusammen absolviert und sind ein eingespieltes Team.

9.1.2 Project Schedule – Version 0

This project schedule was created at the beginning of the project.

Zeitplan PA Drone Tuning

			Semesterwoche													
Week	**Start SW**	**Ende SW**	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Date			15.09	22.09	29.09	6.10	13.10	20.10	27.10	3.11	10.11	17.11	24.11	1.12	8.12	15.12
Description																
Erstellung Zeitplan	1	1														
Testversuche Programm und Drohne	1	3														
Simulation Regler (Simulink/MATLAB)	1	3														
Codestruktur definieren	2	4														
Drohne bauen	3	5														
Milestone Meeting 1	5	5														
MATLAB nach Codestruktur (Klassen) umschreiben	4	7														
Weiterentwicklung MATLAB Code	7	9														
.txt Logfile in .csv umwandeln	9	11														
Milestone Meeting 2	9	9														
MATLAB -> Python	9	12														
Optimierung Chirp Signal Betaflight	12	14														
Doku	10	14														
GitHub Anleitung	10	14														

Abbildung 6: Project Schedule – Version 0

9.1.3 Project Schedule – Version 1

This project schedule was created prior to the first milestone meeting and approved during the meeting.

Zeitplan PA Drone Tuning

			Semesterwoche													
Week	**Start SW**	**Ende SW**	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Date			15.09	22.09	29.09	6.10	13.10	20.10	27.10	3.11	10.11	17.11	24.11	1.12	8.12	15.12
Description																
Erstellung Zeitplan	1	1														
Testversuche Programm und Drohne	1	4														
Simulation Regler (Simulink/MATLAB)	1	3														
Simulation Übertragungsfunktion	2	4														
Codestruktur definieren	2	4														
Simulation PID Regler	3	4														
Simulation Spektrum	4	5														
Drohne bauen	5	7														
Milestone Meeting 1	5	5														
Simulation Spektrogram	5	6														
MATLAB nach Codestruktur (Klassen) umschreiben	5	8														
~~Weiterentwicklung MATLAB Code~~	~~7~~	~~9~~														
.txt Logfile in .csv umwandeln	1	6														
Milestone Meeting 2	9	9														
MATLAB -> Python	8	10														
Weiterentwicklung Python Code	10	13														
Optimierung Chirp Signal Betaflight	12	14														
Doku	10	14														
GitHub Anleitung	10	14														

Abbildung 7: Project Schedule – Version 1

9.1.4 Project Schedule – Version 2

This project schedule was created prior to the second milestone meeting and approved during the meeting.

Zeitplan PA Drone Tuning

			Semesterwoche													
Week	**Start SW**	**Ende SW**	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Date			15.09	22.09	29.09	6.10	13.10	20.10	27.10	3.11	10.11	17.11	24.11	1.12	8.12	15.12
Description																
Erstellung Zeitplan	1	1														
Testversuche Programm und Drohne	1	4														
Simulation Regler (Simulink/MATLAB)	1	3														
Simulation Überttragungsfunktion	2	4														
Codestruktur definieren	2	4														
Simulation PID Regler	3	4														
Simulation Spektrum	4	5														
Drohne bauen	5	12														
Milestone Meeting 1	5	5														
Simulation Spektrogram	5	6														
MATLAB nach Codestruktur (Klassen) umschreiben	5	8														
~~Weiterentwicklung MATLAB Code~~	~~7~~	~~9~~														
.txt Logfile in .csv umwandeln	1	6														
Milestone Meeting 2	9	9														
Erstellung eines Miniskript in Python	7	10														
GitHub Dokumentation	8	11														
Optimierung Codestruktur MATLAB	9	11														
Umschreiben der MATLAB Libary in Python mit Funktionstest	10	13														
~~MATLAB -> Python~~	8	10														
~~Weiterentwicklung Python Code~~	10	13														
~~Optimierung Chirp Signal Betaflight~~	12	14														
Doku	10	14														
GitHub Anleitung	10	14														

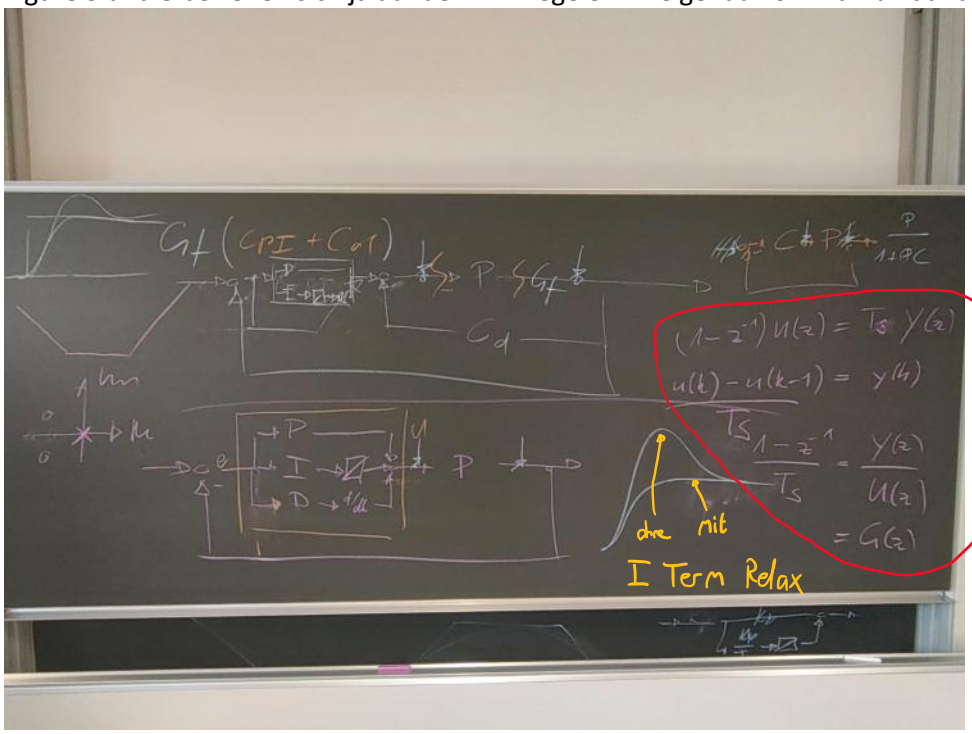
Abbildung 8: Project Schedule – Version 2

9.1.5 Meeting Minutes

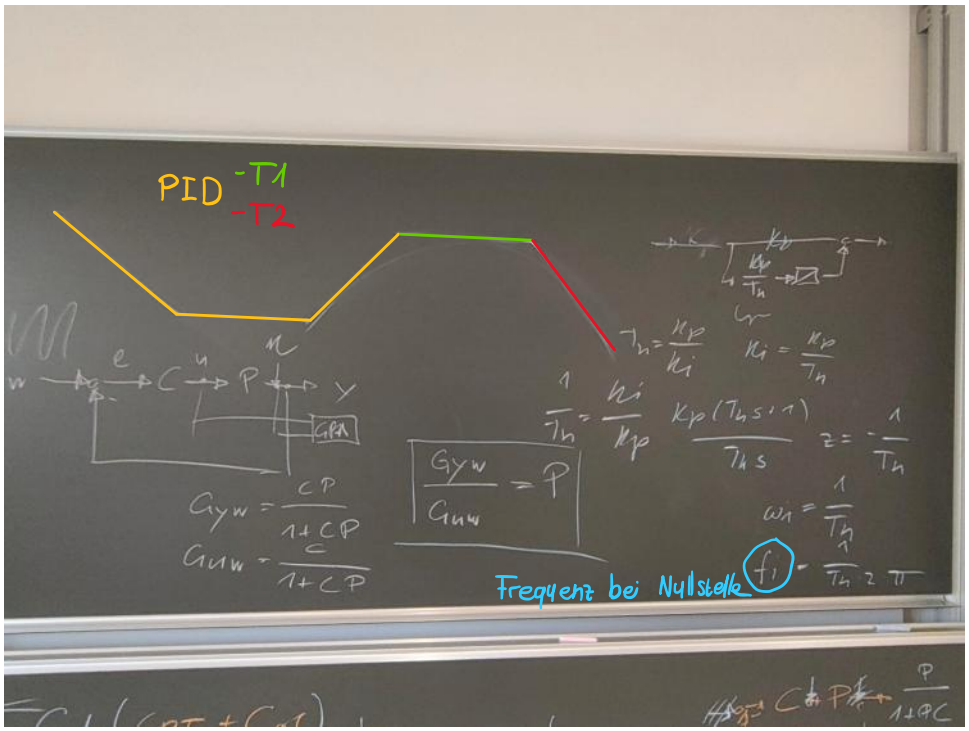
22.09.2025 - Weekly Meeting

Mittwoch, 17. September 2025 11:17

Offene Fragen:

- Figure 5 und 8 beziehen sich ja auf den PID Regeler. Wie genau kommt man auf den gemessenen PID Wert?
- 

- **Simulation**
- Wo genau findet man "get_pid_scale" ? Ist das ein Wert aus der Libary?
- Wieso muss ich die Nachstellzeit/Nachstellfrequenz berechnen? Grundsätzlich wäre es doch auch noch interessant die Nachstellzeit runter oder hochzuschrauben?



29.09.2025 - Weekly Meeting

Montag, 22. September 2025 23:17

Offene Fragen:

- Generelle Weiterentwicklungen wie zum Beispiel das Einsetzen von Slider oder von Auswahlhäckchen, müssen die in Matlab implementiert werden oder würde es in Python reichen?

Simulation / Generell Code:

- Slider oder auswahlhäkchen für vergeich neu/alt. Nur Python?
- Kann man ein Dynamischen Notchfilter oder ein dynamischer Tiefpassfilter Simulieren? Grundsätzlich im Betaflight Code gefunden, erklärung noch von Vorteil.
- was macht der Yaw Tiefpass Filter und wo im Regelkreis ist er zu platzieren? (Betaflight)
- Wieso sieht man nur bei der apex5 Drohne die Eigenfrequenz im Bodeplot?

Yuri

- [ ] Drohne tunen
- [ ] MATLAB Klassenaufteilung

Dario

- [ ] Decodierung

Janick

- [ ] Simulation weiterführen

Yuri

- [ ]

Dario

- [ ]

Janick

- [ ]

02.10.2025 - Weekly Meeting

Dienstag, 30. September 2025 16:52

Offene Fragen:

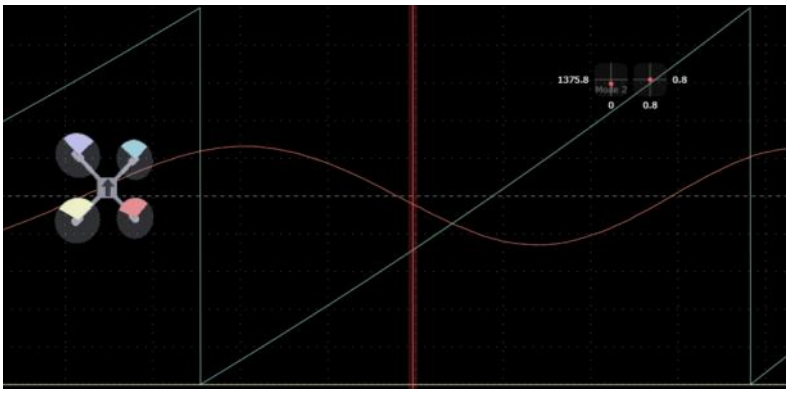
- Diskussion mit ChatGPT, welchen Einfluss auf die Messung hat der Flug vor dem Chirp Signal. ChatGPT meint dass das den Frequenzgang verfälscht. Ist das so? Meiner Meinung nach sollte es eigentlich kein Einfluss haben, da das System haben.
- Könntest du kurz darauf eingehen wie das **Rotate–Filter–Inverse-Rotate**-Verfahren funktioniert?

16.10.2025 - Weekly Meeting

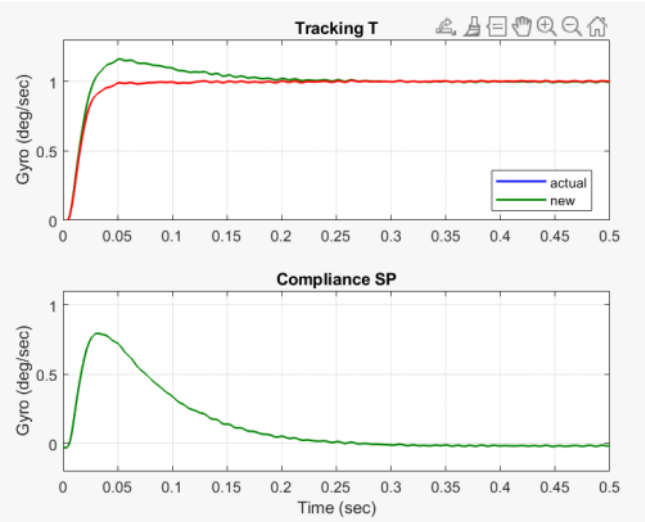
Montag, 6. Oktober 2025 11:05

Milestone Meeting wäre noch offen. Terminfindung
Offene Fragen:

- Gemäss Betaflight Code kann der Biquadfilter eigentlich alles sein, Tiefpassfilter, Hochpassfilter, Bandpassfilter oder Notch Filter. Kann das bei der Filtereinstellung eigentlich sein?
- Wie komme ich aus dem Spektrogramm auf meine Filtereinstellungen im Filter-tab in Betaflight
  - Welche Filter brauche ich überhaupt
  - Welchen typ brauche ich PT1, BIQUAD, PT3
  - Welche Werte trage ich ein
- Warum hast du das Eingangs und Ausgangssignal beide hoch und runter gemischt? Würde es nicht reichen einfach das Ausgangssignal hoch und runter zu mischen? Das Eingangssignal ist ja nicht verrauscht? Weil wir ein Closed-Loop haben oder?
- Wenn ich das Rauschen auf 0 Stelle, dann bekomme ich durch die Rotation Abweichungen. Kann das sein? Oder habe ich etwas falsch gemacht?
- Durch den debug[0] erhält man ja das Signal, welches wir für die Bestimmung wo genau das Chirpsignal ist. Das Signal bei debug[0] sieht einfach sehr komisch aus. Wieso ist das so?



- Für was ist das Downsampeln der "analytical controller transferfunction and convert to frd objects" nötig? Wenn alle Diskretisiert sind sollten wir das doch Weglassen können?
- Wieso verwende ich für den Anti Aliasing Filter nur ein PT1, wäre ein PT2 oder sogar PT3 nicht besser? Wegen weniger Phasenreserve?



Ich glaube ich bin einfach schlecht geflogen, mit pt1 sieht es auch so aus

16.10.2025 - Milestone Meeting

Donnerstag, 16. Oktober 2025 13:04

Angesprochene Punkte Zeitplan
- Farblich Markiert, Rot verschoben, Grün neu

Code:
- Keine Weiterentwicklung in Matlab, alle Weiterentwicklungen in Python
- MyPy anschauen bezüglich Typen
- Vorbereitung Muster Klassen
- Toolboxen finden, damit wir möglichst wenig brauchen
- Langfristige Lösung keine Toolbox

Simulationen:
- Strecke nicht so optimal
    - Pol mehr nacht rechts
- weisses rauschen überall gleich viel energie -> rot filter nicht so geeignet

Ansatz von welch nehmen wir an, dass kein rauschen beim eingang anliegt
-> wie kann man Koherenz bestimmen? (sicher oberes ende von BA verständnis)

- Anfangen die Algorithmen von michi einpflegen
- Wunsch: vergleich zwischen Messung und Simulation
    - Simulation sollte passen wie Faust aufs Auge

Python Portierung:
- Matlab code nicht weiterentwickeln
- wenn in python etwas anders gemacht wird -> auch in MATLAB zurückeinpflegen
- gibt schon ein file bei michi in GitHub
  PES_Board/docs/Solutions/python/serial_stream_pi_controller.jpytr

Ausblick:

Yuri:
- RPM Filter und drohne tunen mit 6S akku
- Codestruktur MATLAB

Dario:
- Header zeugs
- Entscheid am nächste Woche ob weiterverfolgen oder nicht
- Python schauen was für librarys

Janick:
- Simulationen

28.10.2025 - Weekly Meeting

Freitag, 17. Oktober 2025 22:08

Beschlüsse letzte Woche

- Anfangen die Algorithmen von michi einpflegen in Simulationen
- Wunsch: vergleich zwischen Messung und Simulation
  - Simulation sollte passen wie Faust aufs Auge

Python Portierung:

- Matlab code nicht weiterentwickeln
- wenn in python etwas anders gemacht wird -> auch in MATLAB zurückeinpflegen
- gibt schon ein file bei michi in GitHub
  PES_Board/docs/Solutions/python/serial_stream_pi_controller.jpytr

Aufgaben seit letzter Woche:

Yuri:

- RPM Filter und drohne tunen mit 6S akku
- Codestruktur MATLAB

Dario:

- Header zeugs
- Entscheid nächste Woche ob weiterverfolgen oder nicht
- Python schauen was für librarys

Janick:

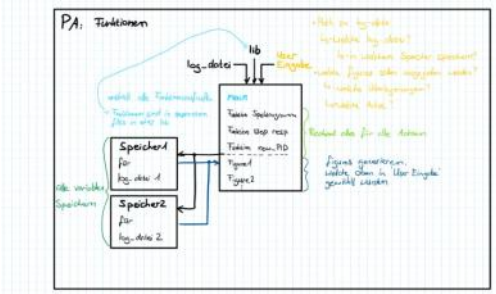
- Simulationen

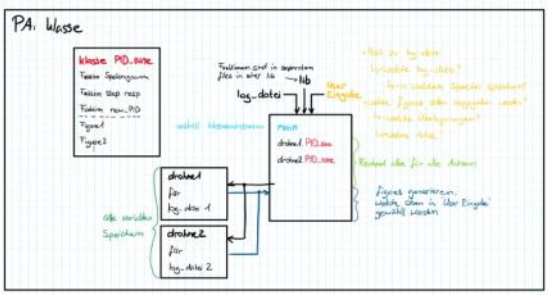
Erledigte Aufgaben

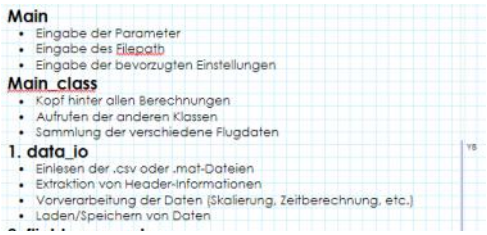
16. Okt	Janick	Mit Anpassungen Regler begonnen. Code neu strukturiert, Daten für Sprungantwort importiert
16. Okt	Yuri, Dario, Janick	Drohnenflug Skills aufgebessert (Flugnachmittag)
16. Okt	Yuri	Begonnen mit ordnerstruktur für strukturierter MATLAB code
17. Okt	Janick	Fertigstellung der Spektogramsimulation
17. Okt	Janick	Import der Funktion aus der Libary für einen Vergleich

20. Okt	Yuri	Codestruktur angepasst und Ordnerstruktur danach in MATLAB erstellt, Packages -> einfacher für Python Portierung begonnen mit Aufteilung
21. Okt	Janick Yuri	Simulation Plant mit Funktion von Michi abgeglichen (Erfolgreich :)) Verucht RPM filter zum laufen zu bringen -> ohne Erfolg... zu alte Firmware und upgrade nicht direkt möglich
21. Okt	Janick	Simulation Spectrum mit Funktion von Michi abgeglichen (Erfolgreich :))
21. Okt	Janick	Simulation Regler mit Funktion von Michi abgegelichen (nicht Erfolgreich :( )
21. Okt	Dario	Identisches header file generiert mit hilfe des blackbox log viewer, Schritt Antwort hat immer noch Schwingungen ☹, sollte möglich sein jedoch unklar welche Werte noch skaliert werden müssen
23. Okt	Yuri	Frequency_response klasse geschrieben und im main eingefügt
23. Okt	Yuri/ Janick	Besprechung wie die Klassen strukturiert werden und wie der Code Aufbau aussehen soll
23. Okt	Dario	Im Source Code von blackbox-log-viewer herumgestöbert um scaling faktoren zu finden, Unterschiede: loopiteratino, time, rxSignal, heading, Flags
24. Okt	Janick	Erstellung des vorläufigen Main und Main_class Teil
26. Okt	Janick	Vorbereitung Weekly Meeting
27. Okt	Janick	Weiterarbeit Klassen und weitere Test dazu
28. Okt	Janick	Herausziehen der Plots, einlesen in möglichkeiten der Codestruktur und auflistung der möglichen Darstellung der Plots mit mehreren Flugdaten
28. Okt	Dario	Blackbox_decode flags angepasst, ANGLE_MODE nun identisch, leider immer noch schwingungen, Verdacht: loopiteration und time sind unterschiedlich wobei das blackbox_tool ein anderes sambling betreibt und so die Daten nicht gleicht manipuliert werden können

Umbau Matlab Code







Main
- Eingabe der Parameter
- Eingabe des Filepath
- Eingabe der bevorzugten Einstellungen

Main_class
- Kopf hinter allen Berechnungen
- Aufrufen der anderen Klassen
- Sammlung der verschiedene Flugdaten

1. data_io
- Einlesen der .csv oder .mat-Dateien
- Extraktion von Header-Informationen
- Vorverarbeitung der Daten (Skalierung, Zeitberechnung, etc.)
- Laden/Speichern von Daten

2. flight_parameters
- Verwaltung von para, para_new, para_used, ind, etc.
- Auswahl des Quads und Achse
- Berechnung und Skalierung von PID-Parametern
- Ausgabe der verwendeten Parameter

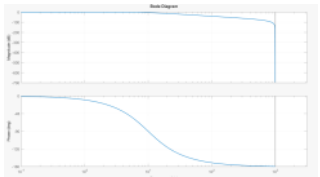
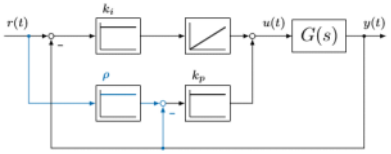
3. spectrogram
- Berechnung von Spektrogrammen
- Ausgabe von Parametern (z. B. Nres, c_lim)
- Visualisierung (z. B. pcolor, log scale, colormap)
- Vergleich gefiltert vs. ungefiltert

4. spectra
- Berechnung von Spektren (FFT, Fensterung, Overlap)
- Ausgabe von Frequenzbereichen
- Visualisierung der Magnituden (logarithmisch)
- Auswahl der relevanten Datenkanäle

5. frequency_response
- Berechnung von Übertragungsfunktionen (T, Guw, Gvw, P)
- Anwendung von Filtern (z. B. apply_rotfiltfilt)
- Visualisierung: Bode-Plots, Coherence
- Vergleich analytisch vs. gemessen

6. controller_analysis
- Berechnung von Cpi, Cd, CL_ana, CL_ana_new
- Closed-loop Analyse (Tracking, Sensitivity, Effort, Compliance)
- Step-Response Berechnung und Darstellung
- Ausgabe der neuen PID-Werte

7. plot_utils
- Gemeinsame Plot-Einstellungen (Farben, Linienbreite, Achsen)
- Funktionen wie linkaxes, expand_multiple_figure_nr
- Standardisierte Achsenlayouts

Offene Fragen
- Bei der estimate_spectrum funktion hat es eine smooth matrix drin. Wie genau Funktioniert diese?
- Bei der Funktion apply_rotfiltfilt muss man noch eine Übertragungsfunktion Glp übergeben. Diese wird dann aber auskommentiert. Für was wäre diese Eigenschaft vorgesehen? Einfach zum hohe Frequenzen rauszufiltern?
- 
- Irgendwas stimmt bei der Berechnung der Übertragungsfunktion im Vergleich zu Simulink nicht. System wird nach Berechnung Instabil aber in Simulink ist es Stabil
- Item Relax, prinzip wie in der Schule gehabt?
- 

Generell Aufgabenteilung:
- Wechsel Arbeiten Dario zu Firmware?

Weiter Schritte:

Janick:
- Weiterführung Matlab Code

Yuri:
- Weiterführung Matlab Code

Dario:
- Arbeit bis jetzt sauber abschliessen
- Csv -> einlesen -> G_cl() herleiten -> sprungantwort -> plotten
- https://github.com/pichim/PES_Board/blob/main/docs/solutions/python/serial_stream_pi_controller.ipynb
- https://github.com/pichim/Filter_Implementation_Test/blob/main/docs/python/eval_filter_implementation.ipynb

03.11.2025 - Weekly Meeting

Freitag, 31. Oktober 2025 23:40

Beschlüsse letzte Woche

- Beendigung der Arbeiten vom direkten Einlesen der Daten aus dem bbl file.
- Weiterarbeiten am Matlab Code mit klassen
- Vorbereitung Python Code

Weiter Schritte:

Janick:

- Weiterführung Matlab Code

Yuri:

- Weiterführung Matlab Code

Dario:

- Arbeit bis jetzt sauber abschliessen
- Csv -> einlesen -> G_cl() herleiten -> sprungantwort -> plotten
- https://github.com/pichim/PES_Board/blob/main/docs/solutions/python/serial_stream_pi_controller.ipynb
- https://github.com/pichim/Filter_Implementation_Test/blob/main/docs/python/eval_filter_implementation.ipynb

Arbeiten in der vergangenen Woche:

28. Okt	Dario	Blackbox_decode branch aufgeräumt für späteren Gebrauch
29. Okt	Janick	Figuren 1-3 in der Klasse plots_utils erstellt
30. Okt	Yuri	Versucht Klasse frequency response estimator zu schreiben
30. Okt	Janick	Figuren 4-6 in der Klasse plot_utils erstellt
30. Okt	Dario	Code etwas genauer angeschaut, geplant extract_header_information umzuschreiben ausser Error Handling momentan nichst zu verbessern.
31. Okt	Yuri	- FC neue Firmware geflasht -> Treiberproblem (STTub30.sys) und FC hatte ein Problem am Anfang - Frequency Response klasse weitergearbeitet -> macht es überhaupt Sinn eine Klasse daraus zu machen? - Codestruktur?
31. Okt	Janick	Figuren 7-9 in der Klasse plot_utils erstellt
1. Nov	Janick	Figuren 22 und 99 in der Klasse plot_utils erstellt
1. Nov	Yuri	Neue Regler und FC in Drohne eingebaut, Modi eingestellt etc.
1. Nov	Janick	Dokumentation von der Funktion apply_rotrotfilt erstellt, Inklusive Bilder.

Probleme und Lösungsvorschläge:

- Anpassung der Codestruktur. Neu mischung Objektorientiert und Prozedual

Fragen:

- Ziel ist ja, das wir die Dokumentation für GitHub erstellen und nur noch eine kleine Version als Abgabe? Wie genau stellst du dir das vor? Sollen wir die Theorie dahinter erklären? (Beispiel apply_rotfiltfilt)
- Wie ist das mit den Quellen? Müssen wir das bei Github auch angeben?

Prio für Doku:

1. User
2. Entwickler

Weitere Schritte:

Dario:

- Simples Matlab-Script erstellen, workflow in python implementieren

Janick:

- Dokumentation weiterführen (nicht zu tief)
- Auskommentieren Matlab Main und Plot Klasse

Yuri:

- 6 Zoll Drohne tunen mit neuen ESC's und neuem FC
- MATLAB Code komentieren
- Anfangen mit Pathon portierung

10.11.2025 - Weekly Meeting

Dienstag, 4. November 2025 10:36

Beschlüsse letzte Woche:

- Zusammenarbeit bei Python
- Dokumentation für Github, Anleitung für Nutzung hat Vorrang vor den Erklärungen für Entwickler

Weiter Schritte letzte Woche:

Dario:

- Simples Matlab-Script erstellen, workflow in python implementieren

Janick:

- Dokumentation weiterführen (nicht zu tief)
- Auskommentieren Matlab Main und Plot Klasse

Yuri:

- 6 Zoll Drohne tunen mit neuen ESC's und neuem FC
- MATLAB Code komentieren
- Anfangen mit Pathon portierung

Arbeiten letzte Woche:

Datum	Wer	Was
3. Nov	Yuri	Flug mit 6 Zoll quad -> log nicht aufgezeichnet -> falsche baudrate...
4. Nov	Yuri	Flug mit 6 Zoll quad -> dieses mal mit logging:)
4. Nov	Janick	Option mit default Filterwerten hinzugefügt
4. Nov	Janick	Dokumentation für " Estimate Frequency Response", "Filter and Controller" und "Sinarg" erstellt
6. Nov	Yuri	- 6 Zoll quad - tuning - MATLAB Code auskommentiert und verschönert - neuer Branch für python erstellt - nachgeschaut, ob es möglich ist eine funktion in python in MATLAB aufzurufen
6. Nov	Janick	Dokumentation für "Estimate Transferfuction Plant" , "Calculation Close Loop" und "Step Response" erstellt
6. Nov	Dario	Mini Script für Stepresponse in Matlab fertiggestellt
9. Nov	Yuri	- files in lib in python umgechrieben und erste tests im MATLAB - 6 Zoll quad neue Einstellungen geladen und bereit für Flug
9. Nov	Dario	Environment für Python aufgesetzt
9. Nov	Dario	Mini Script Stepresponse implementiert

Fragen:

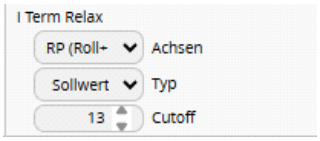
- Ich bekomme immer diese Meldungen im Command Window wenn ich unser Programm ausführe

```
In controllib.chart.internal.foundation/RowColumnPlot/createView (line 899)
In controllib.chart.internal.foundation/AbstractPlot/build (line 4109)
In controllib.chart.internal.foundation/AbstractPlot/cbVisibilityChanged (line 2934)
In controllib.chart.internal.foundation.AbstractPlot>@(es,ed)cbVisibilityChanged(this) (line 1880)
In controllib.chart.internal.utils.ltiplot (line 563)
In DynamicSystem/bodeplot (line 88)
In DynamicSystem/bode (line 102)
In plot_utils/plotCPIDBode (line 387)
In main (line 189)
Warning: Negative limits ignored
> In controllib.chart.internal.layout/LimitManager/updateYLim (line 228)
In controllib.chart.internal.layout/AxesGrid/updateLimits (line 1756)
In controllib.chart.internal.layout/AxesGrid/update (line 491)
In controllib.chart.internal.view.axes/BaseAxesView/set.YLimits (line 641)
In controllib.chart.internal.foundation/AbstractPlot/createView (line 2251)
In controllib.chart.internal.foundation/RowColumnPlot/createView (line 899)
In controllib.chart.internal.foundation/AbstractPlot/build (line 4109)
In controllib.chart.internal.foundation/AbstractPlot/cbVisibilityChanged (line 2934)
In controllib.chart.internal.foundation.AbstractPlot>@(es,ed)cbVisibilityChanged(this) (line 1880)
```

- Habe ich hier ein Bug gefunden? :)

```
% Dynamic gyro lowpass filter 1
if para.gyro_lowpass_dyn_hz(1) > 0
    % Make sure Gf is 1 at start, this is not possible in current bf
    Gf = ss(tf(1, 1, Ts));
    para_used.gyro_lowpass_dyn_hz = para.gyro_lowpass_dyn_hz;
    para_used.gyro_soft_type      = para.gyro_soft_type;
    para_used.gyro_lpf_hz_avg     = get_fcut_from_exp(para.gyro_lowpass_dyn_hz(1), ...
                                                      para.gyro_lowpass_dyn_hz(2), ...
                                                      para.gyro_lowpass_dyn_expo, ...
                                                      throttle_avg);
    para_used.gyro_lpf_throttle_avg = throttle_avg;
    Gf = Gf * get_filter(filter_types{para.dterm_filter_type + 1}, ...
                         para_used.gyro_lpf_hz_avg, ...
                         Ts);
```

-



wie genau wirkt sich den Cutoff auf das signal aus? wird das in deinem Code berücksichtigt?

- Gibt es ein Grund wieso wir den I_ratio definieren und nicht den I-Wert? Fehlerquelle beim Tunen…

Weitere Schritte:

Dario:

- Stepresponse script kondensieren und existierende Funktionen verwenden, in python Funktionen umschreiben

Janick:

- Funktionen in Python umschreiben
- Dokumentation weiterführen

Yuri:

- Funktionen in Python umschreiben
- Flug mit neuen tunes
- I_ratio oder wirklicher I-Wert

17.11.2025 - Weekly Meeting

Montag, 10. November 2025 09:59

Beschlüsse letzte Woche:

- Dario: Stepresponse script kondensieren und existierende Funktionen verwenden, in python Funktionen umschreiben
- Generell: Start umschreiben Matlab zu Python
- Janick: Doku weiterführen (ZHAW Logo kann entfernt werden)
- Übergabe Matlabprogramm an Michi für Test

Weiter Schritte letzte Woche:

Dario:

- Stepresponse script kondensieren und existierende Funktionen verwenden, in python Funktionen umschreiben

Janick:

- Funktionen in Python umschreiben
- Dokumentation weiterführen

Yuri:

- Funktionen in Python umschreiben
- Flug mit neuen tunes
- I_ratio oder wirklicher I-Wert

Arbeiten letzte Woche:

Datum	Wer	Was
11. Nov	Yuri	Step response Funktion in python umgeschrieben und in MATLAB implementiert
11. Nov	Janick	Dokumentation "Chirp Window Selection" erstellt und Erweiterung Sinarg Doku
11. Nov	Janick	I_ration zu I_neu in Matlab umgeschrieben
13. Nov	Yuri	- Spectra und Spectrogram Funktionen in python umgeschrieben und in MATLAB implementiert - Plant, etc Funktionen umgeschrieben aber och nicht in MATLAB implementiert
13. Nov	Dario	Matlab test script mit hilfe der vorhandenen Funktionen auf das Minimum kondensiert
13. Nov	Janick	Besprechnung vorgehen Python Code und start Testen der Funktionen
14. Nov	Dario, Yuri, Janick	Start Drohnenbau
14. Nov	Janick	Python Main Datei erstellt und mit der Gyro_ctrl_tuning Klasse bis Spektogram erstellt
14. Nov	Yuri	Weitergemacht mit Umschreiben in python
15. Nov	Janick	Python Gyro_ctrl_tuning Klasse fertig gestellt
16. Nov	Dario	Python script auf das matlab script angepasst

Besprechung Code Review

Fragen:

- Wie genau stellst du dir die schriftliche Doku für die ZHAW vor?

Weitere Schritte:

Janick:

- MATLAB Code umschreiben gemäss Review
- Weiterführung Dokumentation

Yuri

- Fliegen gehen mit den neuen Tunes
- MATLAB Code umschreiben gemäss Review

Dario

- MATLAB Code umschreiben gemäss Review
- Python Funktionen Testen

24.11.2025 - Weekly Meeting

Dienstag, 18. November 2025 17:22

Beschlüsse letzte Woche:
- Klasse aufspliten mit Richtwert Code Review von Michi
- In Python Funktionen umschreiben, aber mit dem generellen Code noch nicht weiterfahren

Weiter Schritte letzte Woche:
Janick:
- MATLAB Code umschreiben gemäss Review
- Weiterführung Dokumentation

Yuri
- Fliegen gehen mit den neuen Tunes
- MATLAB Code umschreiben gemäss Review

Dario
- MATLAB Code umschreiben gemäss Review
- Python Funktionen Testen

Arbeiten letzte Woche:

Datum	Wer	Was
18. Nov	Yuri	Flug mit neuen tunes :)
18. Nov	Janick	Dokumentation "Convert and evolution Time", "Dataimport" und "Unscale high Resolution Gain" erstellt
19. Nov	Yuri	Start mit der Dokumentation für ZHAW (nicht GitHub)
19. Nov	Dario	Sprungantwort in python verbessert
20. Nov	Janick	Klassen "Flight Data" und "Flight Analyser" erstellt
20. Nov	Dario	Dokumentation für ZHAW
21. Nov	Janick	Klasse "Gyro Ctrl Tuning" erstellt
22. Nov	Yuri	Doku
22. Nov	Janick	Klasse "Plot utils" erstellt
22. Nov	Dario	Python mini-script auf geräumt und besser strukturiert
23. Nov	Yuri	Doku

Fragen:
- Compensate Iterm, wie genau bist du auf diesen Algorithmus gekommen?
- für die Anleitung zum verwenden des tools, können wir die Anleitung aus deinem GitHub übernehmen?

Weitere Schritte:
Janick:
- Optimierung Code
- Erstellung Anleitung Figures

Yuri
- Doku
- weiterer flug mit neuen tunes

Dario
- Doku
- python Funktionen testen

Für PA:
python:
- lib portiert und getestet
- mini python programm

Doku:
- alles markdwon, word oder latex?
- Sachen von michis GitHub können wir übernehmen
- README anpassen

MATLAB:
- besser optimieren
- kommentieren
- Dämpfung zu Q faktor muss noch weg (erledigt)
- 'dono what that is' muss weg
- figures einbauen welche Achse angezeigt wird im Titel (erledigt)

01.12.2025 - Weekly Meeting

Dienstag, 25. November 2025 16:35

Beschlüsse letzte Woche:
Für PA:
python:
- lib portiert und getestet
- mini python programm

Doku:
- alles markdwon, word oder latex?
- Sachen von michis GitHub können wir übernehmen
- README anpassen

MATLAB:
- besser optimieren
- kommentieren
- Dämpfung zu Q faktor muss noch weg (erledigt)
- 'dono what that is' muss weg
- figures einbauen welche Achse angezeigt wird im Titel (erledigt)

Weitere Schritte letzte Woche:
Janick:
- Optimierung Code
- Erstellung Anleitung Figures

Yuri
- Doku
- weiterer flug mit neuen tunes

Dario
- Doku
- python Funktionen testen

Arbeiten letzte Woche:

Datum	Wer	Was
24. Nov	Janick	Neue funktion "Bode_plot_opt" erstellt
25. Nov	Janick	Cutofffrequenz anstatt Dämpfungskonstante bei Notchfilter hinzugefügt
25. Nov	Janick	Figures für beschrieben und wie dadurch die Drohne getunt werden kann
25. Nov	Janick	Dokumentation "Compensate Iterm" erstellt
25. Nov	Dario	Funktionen testen in python
25.Nov	Yuri	Dokumentation
26. Nov	Dario	Funktionen testen in python
27. Nov	Yuri	Dokumentation und Lötstellen beim Drohnenbau nachgebessert
27. Nov	Dario	Funktionen testen in python
29. Nov	Janick	Kontrolle und Anpassungen Tuningbeschrieb
2. Dez	Janick	Überarbeitung Dokumentation und einfügen Bilder

Fragen:
- Bei der Step Response mitteln wir ja die Compliance. Was ist deine Idee dahinter gewesen?
- Abweichungen bei numerischen Funktionen in python, wieviel ist akzeptabel?

Weitere Schritte:
Janick:
- Doku

Yuri:
- Doku

Dario:
- Funktionen testen

9.2 Additional Information

```
clc
clear
close all

s = tf('s');

% Options for Bode-Plots
bodeopts = bodeoptions;
bodeopts.FreqUnits = 'Hz';
bodeopts.MagUnits = 'dB';
bodeopts.Grid = 'on';
bodeopts.PhaseWrapping = 'on';
bodeopts.YLimMode = 'auto';
bodeopts.XLimMode = 'auto';

% Options for Step-Plots
stepopts = timeoptions;
stepopts.Grid = 'on';
stepopts.YLimMode = 'auto';
stepopts.XLimMode = 'auto';

%% =Parameters=======================================

%-Controller-Type----------------------------------
% 0: P
% 1: I
% 2: PI
% 3: PD
% 4: PID
% 5: PID-T1
% 6: PID-T2
controller_type = 4;

%-Kp,-Ki,-Kd-values--------------------------------
Kp = 600;
Ki = 10;
Kd = 100;

% Filterfactor (-> lowpass filter 1. ord.)
% higher N -> closer to ideal
% lower N -> stronger filtering
N = 1000;

%-Plant-Type---------------------------------------
% 0: no Plant
% 1: Integrator
% 2: PT1
% 3: PT2
% 4: Totzeitglied
% 5: Combination: Integrator + PT1
plant_type = 0;
```

```
%% -Controller-----------------------------------------

switch controller_type
    case 0 % P
        C = tf(Kp, 1);
        controller_name = 'P';
    case 1 % I
        C = Ki/s;
        controller_name = 'I';
    case 2 % PI
        C = Kp + Ki/s;
        controller_name = 'PI';
    case 3 % PD
        C = Kp + Kd*s;
        controller_name = 'PD';
    case 4 % PID (ideal)
        C = Kp + Ki/s + Kd*s;
        controller_name = 'PID';
    case 5 % PID-T1 (D-Anteil mit TP1)
        Tf = Kd/N;
        C = Kp + Ki/s + (Kd*s)/(Tf*s + 1);
        controller_name = 'PID-T1';
    case 6 % PID-T2 (D-Anteil mit TP2)
        C = Kp + Ki/s + (Kd*s)/(1 + (Kd/N)*s + (Kd/N)^2 * s^2);
        controller_name = 'PID-T2';
    otherwise
        error('invalid controller type')
end

%% -Plant----------------------------------------------

switch plant_type
    case 0 % Plant off
        P = 1;
    case 1 % Integrator
        P = 1/s;
        plant_name = 'Integrator';
    case 2 % PT1
        T1 = 0.5;
        Kp_strecke = 1;
        P = Kp_strecke / (1 + T1*s);
        plant_name = 'PT1';
    case 3 % PT2
        wn = 10;
        zeta = 0.5;
        P = wn^2 / (s^2 + 2*zeta*wn*s + wn^2);
        plant_name = 'PT2';
    case 4 % dead-time element (Pade-Approximation)
        Tt = 1;
        P = pade(exp(-Tt*s), 4);
        plant_name = 'dead-time element';
    case 5 % Integrator + PT1
        T1 = 0.5;
        P = 1 / (s*(1 + T1*s));
```

```
    otherwise
        error('invalid plant-type')
end

%% -Loop-calculations--------------------------------

L = C * P;            % Open Loop
T = L / (1 + L);      % Closed Loop without prefilter

%% -Plots-----------------------------------------

% Frequency Response Controller
figure(1)
bode(C, bodeopts, 'r')
title(sprintf('Frequency Response \n%s-Controller', controller_name))

if controller_type ~=4 & controller_type ~=3
    % Step Response
    figure(4)
    step(C, stepopts, 'b')
    title(sprintf('Step Response \n%s-Controller', controller_name))
    % draw Line on Y-axis from 0 to start of curve
    [y, t] = step(C);
    hold on
    plot([0 0], [0 y(1)])
    hold off
end

if plant_type ~=0
    % Frequency Response Plant & Open Loop
    figure(2)
    bode(P, bodeopts), hold on
    bode(L, bodeopts)
    legend(sprintf('Plant: %s', plant_name), 'Open Loop L')
    title('Frequency Responses')

    % Nyquist-Diagramm
    figure(3)
    nyquist(L), grid on
    title(sprintf('Nyquist-Diagram Open Loop \n%s-Controller', controller_name))

    % Step Response
    figure(5)
    step(L)
    title(sprintf('Step Response Closed Loop \n%s-Controller', controller_name))
end
```

```
clc, clear variables

%% Bodeplot Settings
% Preferences
set(cstprefs.tbxprefs, 'MagnitudeUnits', 'dB');
set(cstprefs.tbxprefs, 'FrequencyUnits', 'Hz');
set(cstprefs.tbxprefs, 'UnwrapPhase', 'Off');
set(cstprefs.tbxprefs, 'Grid', 'On');
set(groot,'defaultLineLineWidth',1.2);   % global for all new lines

% Bodeoptions
opt = bodeoptions('cstprefs');
opt.Xlim = { [0.1 450] };      % x-axis: 0.1 Hz to 1000 Hz

%% Sampling Time
Ts      = 125 * 1.0e-6;        % Gyro loop
Ts_cntr = 2 * Ts;              % Control loop
Ts_log  = 2 * Ts_cntr;         % Logging loop (not used here, just info)
fs      = 1/Ts;

t = (0:Ts:10).';               % Time vector from 0s to 10s (column vector)
N = length(t);                 % number of samples

% frequency vector in Hz
f = (0:N-1)*(fs/N);

s = tf('s');
z = tf('z');

%% Chirp Signal (logarithmic)
f0 = 0.1; f1 = 500;            % Hz
u = chirp(t, f0, 10, f1, 'logarithmic', 0);   % numerical vector
U_In = timeseries(u, t);       % optional: for Simulink/Scopes

%% Interference Signals

% Input interference (white noise, non-periodic)
A_l = 0.4;
omega_l = 2*pi*10;                      % kept for compatibility, not used
l = A_l * randn(size(t));               % non-periodic input noise
Int_In = timeseries(l, t);              % optional: for Simulink/Scopes

% Output interference (band-limited noise, non-periodic)
A_n = 0.4;
omega_n = 2*pi*20;                      % kept for compatibility, not used
raw_noise = randn(size(t));
[b_n,a_n] = butter(4,[5 200]/(fs/2),'bandpass');  % limit to 5-200 Hz band
n = A_n * filtfilt(b_n,a_n,raw_noise);
Int_Out = timeseries(n, t);             % optional: for Simulink/Scopes

%% Combined Input Signals
% (For MATLAB calculations use numeric vectors; timeseries above only for Simulink)
u_tilde = u + l;
```

```
%% Force everything into column form
[t, u, l, n, u_tilde] = deal(t(:), u(:), l(:), n(:), u_tilde(:));

%% Transfer function Plant
P = (s+10) / ((s + 1)*(s+2));

%% Output Signal
y_tilde  = lsim(P, u_tilde, t);     % system response
y        = y_tilde + n;             % add output interference

%% Plot of Input
figure(1)
subplot(311)
plot(t,u);grid on;
title('Input u(t)');
subplot(312)
plot(t,l);grid on;
title('Input Interference Signal l(t)');
subplot(313)
plot(t,u_tilde);grid on;
title('Input u-tilde(t)');
sgtitle('Combination Input Signals')

%% Plot of Output
figure(2)
subplot(311)
plot(t,y_tilde);grid on;
title('Output y-tilde(t)');
subplot(312)
plot(t,n);grid on;
title('Output Interference Signal n(t)');
subplot(313)
plot(t,y);grid on;
title('Output y(t)');
sgtitle('Combination Output Signals')

%% Plot of Signals
figure(3)
subplot(411)
plot(t,u);grid on;
title('Input u(t)');
subplot(412)
plot(t,u_tilde);grid on;
title('Input u-tilde(t)');
subplot(413)
plot(t,y_tilde);grid on;
title('Output y-tilde(t)');
subplot(414)
plot(t,y);grid on;
title('Output y(t)');
sgtitle('Input and Output Signals')

%% Transferfunction without adjustments (plain FFT)
U    = fft(u);
```

```
Y    = fft(y);
H1   = Y ./ U;

% use only positive frequencies up to Nyquist
H1 = H1(1:floor(N/2));
f  = f(1:floor(N/2));

% create FRD object for bode plot
Hfrd = frd(H1, 2*pi*f);   % bode expects rad/s
figure(4)
h = bodeplot(P, Hfrd, opt);
grid on
legend('G(s) (true)','Estimated (FFT)')
op = getoptions(h);
op.Title.String = '';
setoptions(h, op);
sgtitle('Transfer Function Y(s)/U(s)');

%% Transferfunctions with Signalenergy (single-shot on full record)
Syy = Y .* conj(Y);
Suu = U .* conj(U);
Syu = Y .* conj(U);

th   = 1e-15 * max(Suu);      % get rid of extremely small numbers for numerical errors
H2   = Syu ./ Suu;
H2(Suu < th) = NaN;           % guard
H2 = H2(1:floor(N/2));

% create FRD object for bode plot
figure(5)
h5 = bodeplot(P, Hfrd, opt);
grid on
legend('P(s) (true)','Estimated (FFT)')
op5 = getoptions(h5);
op5.Title.String = '';
setoptions(h5, op5);
sgtitle('Transfer Function Syu/Suu');

%% APPLY_ROTFILTFILT (inline, from existing data)

% Basisband low-pass (zero-phase)
%fc = 5;                                    % adjust if sweep speed changes
%[b_lp, a_lp] = butter(4, fc/(fs/2));

% Phase (sinarg) of the logarithmic chirp from f0,f1 and total duration
Ttot = t(end) - t(1);
r    = f1 / f0;
phi  = 2*pi*f0 * ( Ttot/log(r) ) * ( r.^((t - t(1))/Ttot) - 1 );  % Also know as sinarg

% Phasors
p  = exp(1i*phi);
pc = conj(p);
```

```
% Input/Output signals (use actual plant input & measured output)
X_in  = u - mean(u);
X_out = y - mean(y);

% Rotate forward
in_R  = X_in  .* p;   in_Q  = X_in  .* pc;
out_R = X_out .* p;   out_Q = X_out .* pc;

% Zero-phase low-pass in baseband
% in_R  = filtfilt(b_lp, a_lp, in_R);   in_Q  = filtfilt(b_lp, a_lp, in_Q);
% out_R = filtfilt(b_lp, a_lp, out_R);  out_Q = filtfilt(b_lp, a_lp, out_Q);

% Back-rotate & recombine (real)
inp = real(0.5*(in_R .* pc + in_Q .* p));      % demodulated, filtered input
out = real(0.5*(out_R.* pc + out_Q.* p));      % demodulated, filtered output

%% Simple FRF from rotated signals via FFT (optional quick look)
U_rotFFT = fft(inp);
Y_rotFFT = fft(out);
H_rotFFT = Y_rotFFT ./ U_rotFFT;
H_rotFFT = H_rotFFT(1:floor(N/2));
f_axis   = (0:floor(N/2)-1)*(fs/N);
Hfrd_rotFFT = frd(H_rotFFT, 2*pi*f_axis);

figure(9)
p9 = bodeplot(P, Hfrd_rotFFT, opt);
grid on
legend('P(s) (true)','Estimated (FFT, rotated)')
op9 = getoptions(p9); op9.Title.String = ''; setoptions(p9, op9);
sgtitle('Transfer Function (FFT on rotated signals)');

%% Transfer function according to Welch (original & rotated via function)

% Use same parameters you already used above
Nest    = round(2 / Ts);        % number of samples per segment
overlap = 0.9;                  % 50% overlap

% Welch FRF for original signals u -> y
[Hfrd_welch, ~] = welch_frf(u, y, Nest, overlap, fs, true);

% Welch FRF for rotated (Lock-in) signals inp -> out
[Hfrd_welch_rot, ~] = welch_frf(inp, out, Nest, overlap, fs, true);

% Plots (kept consistent with your style)
figure(6); clf
p6 = bodeplot(P, Hfrd_welch, Hfrd, opt);
grid on
legend('P(s) (true)', 'Welch Signal Energie', 'Signal Energie', ...
       'Location','SouthWest');
title('Transfer Function (Original, Welch)');


figure(7)
```

```
p7 = bodeplot(P, Hfrd_welch_rot, Hfrd_rotFFT, opt);
grid on
legend('P(s) (true)', 'Welch Signal (rotated)', 'Signal rotated', ...
       'Location','SouthWest');
op7 = getoptions(p7); op7.Title.String = ''; setoptions(p7, op7);
sgtitle('Transfer Function (Rotated/Lock-in, Welch)');

figure(10)
p10 = bodeplot(P, Hfrd_welch, Hfrd_welch_rot, opt);
grid on
legend('P(s) (true)', 'Welch H_2 (orig)', 'Welch H_2 (rotated)', ...
       'Location','SouthWest');
op10 = getoptions(p10); op10.Title.String = ''; setoptions(p10, op10);
sgtitle('Welch: Original vs Rotated');

%% Welch fuction
function [Hfrd_welch, freq] = welch_frf(u_sig, y_sig, Nest, overlap, fs, use_hann)
% WELCH_FRF  Estimate FRF via Welch method (H2 = S_yu / S_uu) and coherent mean↙
(Y/U).
% Inputs:
%   u_sig   : input signal (column vector)
%   y_sig   : output signal (column vector)
%   Nest    : samples per segment
%   overlap : fraction [0..1) of overlap (e.g., 0.5)
%   fs      : sampling frequency [Hz]
%   use_hann: if true, use Hann window; else rectangular
%
% Outputs:
%   Hfrd_welch : FRD object of H2 estimator
%   Hfrd_coh   : FRD object of coherent estimator mean(Y/U)
%   freq       : frequency vector [Hz] (0..Nyquist)

    delta = 0 * var(u_sig);      % small regularization

    u_sig = u_sig(:); y_sig = y_sig(:);     %Convertion from row vector to column↙
vector

    if use_hann                             % Decide witch window will be used
        w = hann(Nest,'periodic');
    else
        w = ones(Nest,1);
    end

    Ndata = size(u_sig, 1);
    Noverlap = floor(overlap * Nest);


    % When you apply a window (like a Hann window), the signal energy is reduced
    % This the factor to compensate that
    denom = sum(w) / Nest / 2;

    % frequency axis (Hz) and single-sided length
    freq = (0:Nest/2).' * (fs / Nest);
    Nfreq = length(freq);
```

```
% global mean removal
u_sig = u_sig - mean(u_sig);
y_sig = y_sig - mean(y_sig);

% Welch average of single-sided spectra [Suu, Syu, Syy]
Pavg = zeros(Nfreq, 3);
Navg = 0;

ind_start = 1;
ind_end   = Nest;
Ndelta    = Nest - Noverlap;


while ind_end <= Ndata
    ind = ind_start:ind_end;

    u_inp = u_sig(ind);
    y_out = y_sig(ind);

    % per-segment mean removal
    u_seg = u_inp - mean(u_inp);
    y_seg = y_out - mean(y_out);

    % window
    u_seg = u_seg .* w;
    y_seg = y_seg .* w;

    % FFT with single-sided normalization
    U = fft(u_seg) / (Nest * denom);
    Y = fft(y_seg) / (denom* Nest);

    % two-sided -> single-sided (0..Nyquist), then fix DC/Nyquist
    Pact = [U.*conj(U), Y.*conj(U), Y.*conj(Y)];
    P1s  = Pact(1:Nfreq, :);
    P1s(1,  :) = P1s(1,  :) / 4;
    P1s(end,:) = P1s(end,:) / 4;

    Pavg = Pavg + P1s;
    Navg = Navg + 1;

    % Next segment
    ind_start = ind_start + Ndelta;
    ind_end   = ind_end   + Ndelta;
end

if Navg > 0
Pavg = Pavg / Navg;
end

Suu = Pavg(:,1);
Syu = Pavg(:,2);

H_welch = Syu ./ (Suu + delta);  % H2 estimator with regularization
```

```
    Hfrd_welch = frd(H_welch, freq, 1/fs, 'Units','Hz');
end



%% Add function form Michi


addpath ../lib/

% Linear filter for zero phase excitation filter (apply_rotfiltfilt)
Dlp = sqrt(3) / 2;
wlp = 2 * pi * 10;
Glp = c2d(tf(wlp^2, [1 2*Dlp*wlp wlp^2]), Ts_log, 'tustin');


% Lock-in clean-up (rotate, zero-phase filter in baseband, back-rotate)
u_rf = apply_rotfiltfilt(Glp, phi, u);    % filtered input
y_rf = apply_rotfiltfilt(Glp, phi, y);    % filtered output

% Welch parameters (use same scale as your script; ensure even Nest)
Nest    = round(4 / Ts);         % number of samples per segment
NoverlapS = round(0.9 * Nest);         % 90% overlap
win       = hann(Nest,'periodic');
delta     = 1e-12*var(u);            % tiny regularization for Suu

% FRF of Variant C (function-based)
[G_C, C_C, freq_Hz, ~] = estimate_frequency_response(u_rf, y_rf, win, NoverlapS,↙
Nest, Ts, delta);

% --- Bode overlay: true P(s) vs. Original (Welch) vs. Variant C ---
figure(11)
pC = bodeplot(P, Hfrd_welch_rot, G_C, opt); grid on
legend('P(s) (true)','Janick','Michi', ...
       'Location','SouthWest');
opC = getoptions(pC); opC.Title.String = ''; setoptions(pC, opC);
sgtitle('Janick vs Michi');
```

```
%% Simulation Frequenzgang Drohen

clc, clear variables
addpath ../lib/
addpath ../20250907/
s = tf('s');

%% Laden von Drohnenflüge als Referenz
flight_folder = '20250907';

quad = 'apex5';
log_name = '20250907_apex5_00.bbl.csv';

% Choose an axis: 1: roll, 2: pitch, 3: yaw
ind_ax = 1;

%% Daten Laden,

file_path = fullfile(flight_folder, log_name);
[para, Nheader, ind, ind_cntr] = extract_header_information(file_path);

% Read the data
%  - If its the first time from the .csv and save a mat, otherwise the
%    .mat. This increases load speed significantly.
tic
try
   load([file_path(1:end-8), '.mat'])
catch exception
   data = readmatrix(file_path, 'NumHeaderLines', Nheader);
   save([file_path(1:end-8), '.mat'], 'data');
end
[Ndata, Nsig] = size(data);
toc
% Expand index
ind.axisSumPI = ind_cntr + (1:3);
ind.sinarg = ind.debug(1);

%% Einstellungen Bodeplot

% Defines
set(cstprefs.tbxprefs, 'MagnitudeUnits', 'dB');
set(cstprefs.tbxprefs, 'FrequencyUnits', 'Hz');
set(cstprefs.tbxprefs, 'UnwrapPhase', 'Off');
set(cstprefs.tbxprefs, 'Grid', 'On');
set(groot,'defaultLineLineWidth',1.2);   % global für alle neuen Linien
set(0, 'defaultAxesColorOrder', get_my_colors);

% Bodeoptions
opt = bodeoptions('cstprefs');
opt.Xlim = { [0.5 1e3] };      % x-Achse: 0.1 Hz bis 1000 Hz



%% Filterparameter
```

```
%Sampletime
Ts = para.looptime * 1.0e-6;              % Gyro loop
z = tf('z', Ts);

%Drohnenparameter

% type: 0: PT1, 1: BIQUAD, 2: PT2, 3: PT3
para_new.gyro_lpf            = 0;       % dono what this is
para_new.gyro_lowpass_hz     = 0;       % frequency of gyro lpf 1
para_new.gyro_soft_type      = 0;       % type of gyro lpf 1
para_new.gyro_lowpass_dyn_hz = [0, 0];  % dyn gyro lpf overwrites gyro_lowpass_hz
para_new.gyro_lowpass2_hz    = 800;     % frequency of gyro lpf 2
para_new.gyro_soft2_type     = 0;       % type of gyro lpf 2
para_new.gyro_notch_hz       = [0, 520]; % frequency of gyro notch 1 and 2
para_new.gyro_notch_cutoff   = get_fcut_from_D_and_fcenter([0.00, 0.15], para_new.↙
gyro_notch_hz); % damping of gyro notch 1 and 2
para_new.dterm_lpf_hz        = 0;       % frequency of dterm lpf 1
para_new.dterm_filter_type   = 0;       % type of dterm lpf 1
para_new.dterm_lpf_dyn_hz    = [0, 0];  % dyn dterm lpf overwrites dterm_lpf_hz
para_new.dterm_lpf2_hz       = 130;     % frequency of dterm lpf 2
para_new.dterm_filter2_type  = 3;       % type of dterm lpf 2
para_new.dterm_notch_hz      = 235;     % frequency of dterm notch
para_new.dterm_notch_cutoff  = get_fcut_from_D_and_fcenter(0.15, para_new.↙
dterm_notch_hz); % damping of dterm notch
para_new.yaw_lpf_hz          = 200;     % frequency of yaw lpf (pt1)
switch ind_ax
    case 1 % roll: [49, 83, 33, 0]
        P_new       = 1.0 * 49;
        I_ratio_new = 1.0 * 83/83;
        D_new       = 1.0 * 33;
    case 2 % pitch: [61, 103, 39, 0]
        P_new       = 1.0 * 61;
        I_ratio_new = 1.0 * 103/103;
        D_new       = 1.0 * 39;
    case 3 % yaw: [42, 104, 3, 0]
        P_new       = 1.0 * 42;
        I_ratio_new = 1.0 * 104/104;
        D_new       = 1.0 * 3;
end


%% Funktion für Tiefpassfilter
function G = Tiefpassfilter(type, omega, s)
    if omega == 0
        G = tf(1);
    else
        switch type
            case 0 % PT1
                G = 1 / (1 + s/omega);
            case 1 % BIQUAD
                G = 1 / (1 + s/omega); %unklar was für eine Übertrafungsfunktion↙
BIQUAD ist
            case 2 % PT2
```

```
                c = 1.553773974;
                G = 1 / (1 + s/(omega*c))^2;
            case 3 % PT3
                c = 1.961459177;
                G = 1 / (1 + s/(omega*c))^3;
            otherwise
                error('Unbekannter Filtertyp');
        end
    end
end

%% Funktionen diskretisierter Tiefpass
function G = lpf_pt1_discrete(fc, Ts)
    Om = 2*pi*fc*Ts;             % cut off frequency
    k  = Om/(1+Om);              % Steplenght
    G  = tf(k, [1 -(1-k)], Ts); %Transferfunction
end

function G = lpf_discrete(type, fc, Ts)
    if fc == 0      %in case LPF is not activatet
        G = tf(1,1,Ts);
        return;
    end
    switch type
        case 0  % PT1
            G = lpf_pt1_discrete(fc, Ts);

        case 2  % PT2 = PT1 × PT1, mit Cutoff-Correction
            c = 1.553773974;
            H1 = lpf_pt1_discrete(fc*c, Ts);
            G  = H1*H1;

        case 3  % PT3 = PT1 × PT1 × PT1, mit Cutoff-Correction
            c = 1.961459177;
            H1 = lpf_pt1_discrete(fc*c, Ts);
            G  = H1*H1*H1;

        case 1  % BIQUAD: ohne Q keine eindeutige Spec; Placeholder = PT1
            warning('BIQUAD ohne Q: ersetze provisorisch durch PT1.');
            G = lpf_pt1_discrete(fc, Ts);

        otherwise
            error('Unbekannter Filtertyp');
    end
end


%% Gyro-Lowpassfilter

%Calculation LPF1
%Continuous Frequency response
omega1=2*pi*para_new.gyro_lowpass_hz;
G_LPF1 = Tiefpassfilter(para_new.gyro_soft_type, omega1, s);
%Discrete time Frequency response
```

```
G_LPF1_dis  = lpf_discrete(para_new.gyro_soft_type,  para_new.gyro_lowpass_hz,  Ts);

%Calculation LPF2
%Continuous Frequency response
omega2 = 2*pi*para_new.gyro_lowpass2_hz;
G_LPF2 = Tiefpassfilter(para_new.gyro_soft2_type, omega2, s);
%Discrete time Frequency response
G_LPF2_dis  = lpf_discrete(para_new.gyro_soft2_type, para_new.gyro_lowpass2_hz, Ts);

%Calculation LPF1 D-share
%Continuous Frequency response
omegad1 = 2*pi*para_new.dterm_lpf_hz;
G_LPFD1 = Tiefpassfilter(para_new.dterm_filter_type, omegad1, s);
%Discrete time Frequency response
G_LPFD1_dis = lpf_discrete(para_new.dterm_filter_type,  para_new.dterm_lpf_hz,  Ts);

%Calculation LPF2 D-share
%Continuous Frequency response
omegad2 = 2*pi*para_new.dterm_lpf2_hz;
G_LPFD2 = Tiefpassfilter(para_new.dterm_filter2_type, omegad2, s);
%Discrete time Frequency response
G_LPFD2_dis = lpf_discrete(para_new.dterm_filter2_type, para_new.dterm_lpf2_hz, Ts);

figure(1)
bode(G_LPF2,G_LPF2_dis, opt);grid on;
legend('Continuous', 'Discrete','Location', 'southwest');
title('Lowpassfilter 2');


figure(2)
bode(G_LPFD2,G_LPFD2_dis, opt);grid on;
legend('Continuous', 'Discrete','Location', 'southwest');
title('Lowpassfilter D-Term 2');

%% Funktion für Notch-Filter
function G_notch = Notch(f0, fcut, s)
    if f0 ~= 0
        f2 = f0^2 / fcut;
        Q  = f0 / (f2-fcut);           % Calculation with cutoff frequency
        omega = 2*pi*f0;
        G_notch = (s^2 + omega^2) / (s^2 + (omega/Q)*s + omega^2);
    else
        G_notch = tf(1);
    end
end

%% Discretization of Notch filter
function G_notch_dis = Notch_dis(f0, fcut, Ts, z)
    if f0 ~= 0
        f2 = f0^2 / fcut;
        Q  = f0 / (f2-fcut);           % Calculation with cutoff frequency
        omega = 2*pi*f0*Ts;
        alpha = (1 - sin(omega)) / (2*Q);
       G_notch_dis = (1 - 2*cos(omega)*z^-1 + z^-2) ...
```

```
            / ((1 + alpha) - 2*cos(omega)*z^-1 + (1 - alpha)*z^-2);
    else
        G_notch_dis = tf(1);
    end
end

%% Notch Filter berechnung

%Berechnung Notchfilter 1
%Continuous Frequency response
G_Notch1 = Notch(para_new.gyro_notch_hz(1), para_new.gyro_notch_cutoff(1), s);
%Discrete time Frequency response
G_Notch1_dis = Notch_dis(para_new.gyro_notch_hz(1), para_new.gyro_notch_cutoff(1), ↙
Ts, z);

%Berechnung Notchfilter 2
%Continuous Frequency response
G_Notch2 = Notch(para_new.gyro_notch_hz(2), para_new.gyro_notch_cutoff(2), s);
%Discrete time Frequency response
G_Notch2_dis = Notch_dis(para_new.gyro_notch_hz(2), para_new.gyro_notch_cutoff(2), ↙
Ts, z);

%Berechnung Notchfilter D-Anteil
%Continuous Frequency response
G_NotchD = Notch(para_new.dterm_notch_hz, para_new.dterm_notch_cutoff, s);
%Discrete time Frequency response
G_NotchD_dis = Notch_dis(para_new.dterm_notch_hz, para_new.dterm_notch_cutoff, Ts, ↙
z);

figure(3)
bode(G_Notch2, G_Notch2_dis,opt); grid on;
legend('Continuous', 'Discrete','Location', 'southwest');
title('Notchfilter 2');

figure(4)
bode(G_NotchD, G_NotchD_dis, opt); grid on;
legend('Continuous', 'Discrete','Location', 'southwest');
title('Notchfilter D-Term 2');


%% Get old PID values

% PID parameters
    pid_axis = {'rollPID', 'pitchPID', 'yawPID'};
    if (length(para.(pid_axis{ind_ax})) == 5)
        if (para.(pid_axis{ind_ax})(3) ~= para.(pid_axis{ind_ax})(4) && ...
                para.(pid_axis{ind_ax})(4) ~= 0)
            warning([pid_axis{ind_ax}, ' different D gains']);
        end
        % Remove dynamic D-Term
        para.(pid_axis{ind_ax}) = para.(pid_axis{ind_ax})([1 2 3 5]);
    end
    if para.(pid_axis{ind_ax})(4) ~= 0
        warning([pid_axis{ind_ax}, ' FF is not zero']);
```

```
    end
    % Insert 0 for FF
    PID = para.(pid_axis{ind_ax}) .* [get_pid_scale(ind_ax), 0];


%% PID Vektor

pid_scale = [get_pid_scale(ind_ax), 1];

PID_new(1) = P_new * pid_scale(1);                  % Proportionalanteil

%KI von KP abhängig, daher rückrechnung nötig
fI = PID(2) / (2 * pi * PID(1));        % alte Integralfrequenz aus altem PID
fI_new = fI * I_ratio_new;              % gewünschte skalierte Integralfrequenz

PID_new(2) = 2 * pi * PID_new(1) * fI_new;          % Integralanteil
%Differentialanteil unabhängig
PID_new(3) = D_new * pid_scale(3);                  % Differentialanteil
PID_new(4) = 0;

%% Ausgabe PI und D Regler

C_PI = PID_new(1) + PID_new(2)/s;   %PI Regler ohne Filter
Kp = tf(PID_new(1));                %Für Simulink
C_PI_LFP_Notch = C_PI * G_LPF1 * G_LPF2 * G_Notch1 * G_Notch2;     %PI Regler mit↙
Filter

C_D = PID_new(3)*s;     %D Regler ohne Filter
C_D_LFP = C_D*G_LPFD1*G_LPFD2;
C_D_LFP_Notch = C_D* G_LPFD1*G_LPFD2*G_NotchD;      %D Regler mit Filter

%% PID Vektor Diskretisiert

% Calculation of discretization of PI Part of the Controller

C_I_dis = (PID_new(2)*Ts) / (1 - z^-1);   % Calculation I Part
C_PI_dis = PID_new(1) + C_I_dis;          % Combination P and I Part of Controller
C_PI_LFP_Notch_dis = C_PI_dis * G_LPF1_dis * G_LPF2_dis;     %PI Controller with LPF

%Calculation of D Part of the Controller
C_D_dis = (PID_new(3)*(1 - z^-1))/Ts;     %Calculation D Part
C_D_LFP_dis = C_D_dis*G_LPFD1_dis*G_LPFD2_dis;  %D Part with LPF

figure(5)
bode(C_PI, C_PI_dis, C_D_LFP, C_D_LFP_dis, opt);grid on;
legend('Continuous PI', 'Discrete PI', 'Continuous D', 'Discrete D', ...
       'Location', 'southeast');
title('PI and D with LPF Controller');

%% Simulation Strecke P

switch ind_ax
    case 1  %Roll
        %Relevante Frequenzen
```

```
        w2 = 13*2*pi();
        w1 = 16*2*pi();
        wt = 60*2*pi();

        %Übertragungsfunktionen abgeschätzt
        G1 = 1 / (s/w1);
        G2 = 1 / (1 + (s/w2));
        Gt = exp(-s * (1/wt));      %Totzeit
        P_ges = G1*G2*Gt;           %Gesamtübertragungsfunktion
    case 2  %Pitch
        %Relevante Frequenzen
        w2 = 10*2*pi();
        w1 = 13*2*pi();
        wt = 60*2*pi();

        %Übertragungsfunktionen abgeschätzt
        G1 = 1 / (s/w1);
        G2 = 1 / (1 + (s/w2));
        Gt = exp(-s * (1/wt));      %Totzeit
        P_ges = G1*G2*Gt;           %Gesamtübertragungsfunktion
    case 3  %Yaw
        %Relevante Frequenzen
        w2 = 60*2*pi();
        w1 = 10*2*pi();
        wt = 60*2*pi();

        %Übertragungsfunktionen abgeschätzt
        G1 = 1 / (s/w1);
        G2 = 1 / (1 + (s/w2));
        Gt = exp(-s * (1/wt));      %Totzeit
        P_ges = G1*G2*Gt;           %Gesamtübertragungsfunktion
end
P_ges_d = c2d(P_ges, Ts, 'thiran');   % Discretisation of Plant

%% Real Plant
addpath ../Simulations
load ('real_plant_apex.mat');       %Get Data from bf_controller_tuning

figure(6)
bode(P/Gf_ana , P_ges_d, opt);
legend('Measured','Simulated','Location','southeast');
title('Plant, Measured and Simulated');

%% Simulatet Transferfunction
%Continuous Frequency response
% Inner Open loop
G_oli = P_ges * G_Notch1 * G_Notch2 * G_LPF1 * G_LPF2;
G_cli = G_oli / (1+G_oli*G_NotchD*G_LPFD1*G_LPFD2);

%Outer Loop
G_olo = G_cli * C_PI;
G_olc = G_olo / (1+ G_olo);

% Discret Frequency response
```

```
% Inner Open loop
G_oli_dis = P_ges_d * G_Notch1_dis * G_Notch2_dis * G_LPF1_dis * G_LPF2_dis;
G_cli_dis = G_oli_dis / (1+G_oli_dis*G_NotchD_dis*G_LPFD1_dis*G_LPFD2_dis);

%Outer Loop
G_olo_dis = G_cli_dis * C_PI_dis;
G_olc_dis = G_olo_dis / (1+ G_olo_dis);


%% Step response Drone
figure(8)
plot(step_time, step_resp_pl), grid on
xlim([0 , 0.5])

%Step does not work rigth at the moment
figure(9)
step(G_olc_dis);
```

```
clc; clear variables

%% Sampling Time
Ts      = 125 * 1.0e-6;         % Gyro loop
Ts_cntr = 2 * Ts;               % Control loop
Ts_log  = 2 * Ts_cntr;          % Logging loop (not used here, just info)
fs      = 1/Ts;

t = (0:Ts:10).';                % Time vector from 0s to 10s (column vector)
N = length(t);                  % number of samples

%% Thrust
w_thr = 5 * 2*pi;
y = 2 + sin(w_thr*t);

%% Input Signal
omega = 10.1 * 2*pi();
u = 0.6*sin(2*pi*(10 + 1*t).*t) + 0.25*randn(size(t));    %Input signal
w = hann(N, 'periodic');          % window over the entry singal
u_win = u .* w;                   % signal with window

figure(1)
subplot(311)
plot(t,u);grid on;
xlim([0 2]);
xlabel('Time [s]');
title('Input Signal');

subplot(312)
plot(t,u_win);grid on;
xlim([0 2]);
xlabel('Time [s]');
title('Input Signal with Window');

subplot(313)
plot(t, y);grid on;
xlim([0 2]);
xlabel('Time [s]');
title('Thrust');

%% Spectrogram over Time and Thrust

seg = 60;             %Number of segments
Nres = seg;
Nest = floor(N / (1+(seg-1)));  %Number of points in segment
freq(1,:) = (0:Nest/2).' * (fs / Nest);  %Get steplenght of Frequency
Nfreq = length(freq);       %Get Number of Niquist
window = hann(Nest, 'periodic');        % Window over signal

% Creation of y vector
y_min = min(y);
y_max = max(y);
dy    = (y_max - y_min) / Nres;     %Span width of steps
y_axis = (y_min:dy:y_max-dy).';     %Creation of y vector
```

```
u_seg  = zeros(Nest, seg);      %Preparation array
t_seg  = zeros(Nest, seg);      %Preparation array
U_SEG  = zeros(Nest, seg);
U_mean  = zeros(seg);

start = 0;
stop = 0;
y_mid = zeros(seg,1);   % Thrust-Mittelwert je Segment

for i = 1:seg
    start = (i-1)*Nest + 1;
    stop  = start + Nest - 1;

    % Get y vektor
    y_seg = y(start:stop);      % Split up in Segments

    ind_y = round((y_seg - y_min) / max(y_max - y_min, eps) * (Nres - 1)) + 1;   %↙
Split it up in different thrustlevels
    ind_y = max(1, min(Nres, ind_y));   % If thrust is under one get rid off it
    ind_y = sort(ind_y);    % Sort ind_y

    ind_y_count = zeros(size(ind_y));   % Get counter
    j = 1;
    for k = 1:length(ind_y)     % Get throw all ind y
        if ind_y(k) ~= ind_y(j) % Count how many y are the same
            j = j + 1;
            ind_y(j) = ind_y(k);
        end
        ind_y_count(j) = ind_y_count(j) + 1;
    end

    if stop > N
        break;   % Stop when segments are full
    end
    y_mid(i) = mean(y(start:stop)); % Get mean of segment
    u_seg(:, i) = u(start:stop);    % Add Segment to array
    t_seg(:, i) = t(start:stop);
    U_SEG(:,i) = abs(fft(u_seg(:,i)));      % Absolut value of frequent component

end
% Get the step length of the t vector
t_mid = ((0:seg-1)*Nest + (Nest-1)/2) * Ts;   %

% only positiv frequencies
U_POS    = U_SEG(1:Nfreq, :);                   % Ged rid of Alising

figure(2)
pcolor(freq, t_mid, U_POS.');
shading flat; set(gca,'ColorScale','log'); colormap jet; colorbar
xlabel('Frequency (Hz)'); ylabel('Time (s)'); title('Amplitude over Frequency and↙
Time');
xlim([0 100]);
```

```
figure(3)
pcolor(freq, y_mid, U_POS.');
shading flat; set(gca,'ColorScale','log'); colormap jet; colorbar
xlabel('Frequency (Hz)'); ylabel('Thrust');
title('Amplitude over Frequency and Thrust');
xlim([0 100]); ylim([0 3])


%% Get Spectrum with funciton

addpath ../lib/
```

```
clc, clear variables

%% Sampling Time
Ts      = 125 * 1.0e-6;         % Gyro loop
Ts_cntr = 2 * Ts;               % Control loop
Ts_log  = 2 * Ts_cntr;          % Logging loop (not used here, just info)
fs      = 1/Ts;

t = (0:Ts:10).';                % Time vector from 0s to 10s (column vector)
N = length(t);                  % number of samples

%% Input Signal
omega = 10.1 * 2*pi();
u = 0.6*sin(2*pi*(10 + 0.5*t).*t) + 0.25*randn(size(t));            %Input signal
w = hann(N, 'periodic');          % window over the entry singal
u_win = u .* w;                   % signal with window

figure(1)
subplot(211)
plot(t,u);grid on;
title('Input Signal without Window');

subplot(212)
plot(t,u_win);grid on;
title('Input Sigal with Window');

%% Spectrum of Input Signal

U = fft(u);
U_WIN = fft(u_win);
w_fac = sum(w) / 2;

% Frequenzachse (Hz)
f = (0:N-1)*(fs/N);

% Calculation of spectra
A_noWin = abs(U(1:N/2)) / (N/2);     % New vector without Niquist
A_win   = abs(U_WIN(1:N/2)) / w_fac;    %New vector with upscaling because of win
f_half  = f(1:N/2);                  % New vector without Niquist

% Plot
figure(2)
subplot(2,1,1)
plot(f_half, A_noWin)
xlabel('Frequenz [Hz]')
ylabel('Amplitude')
title('Spektrum ohne Fenster')
grid on
xlim([0 100])

subplot(2,1,2)
plot(f_half, A_win)
xlabel('Frequenz [Hz]')
ylabel('Amplitude')
```

```
title('Spektrum mit Hann-Fenster')
grid on
xlim([0 100])

%% Split up in Segments

seg = 7;            %Number of segments
overlap = 0.5;      %Overlap in next segement x*100%
Nest = floor(N / (1+(seg-1)*(1-overlap)));  %Number of points in segment
window = hann(Nest, 'periodic');        % Window over signal
Noverlap = round(overlap * Nest);       % overlap in SAMPLES for function + hop calc
Nshift   = Nest - Noverlap;             % hop size
freq = (0:Nest/2) * (fs / Nest);        %Niquistfrequence
Nfreq = length(freq);                   %Number of steps to Niquist
W = sum(window) / Nest / 2;


Nsignals = 1;
Pavg = zeros(Nfreq, Nsignals);
Pavgw = zeros(Nfreq, Nsignals);

Navg = 0;       % Counter of loop
for i = 1:seg

    start = (i-1)*Nshift + 1;
    stop  = start + Nest - 1;
    if stop > N
        break;   % Stop when segments are full
    end
    u_seg = u(start:stop);           % Fill up segment
    u_seg = u_seg - mean(u_seg);     % per-segment DC removal

    u_seg_win = u_seg .* window;     % Lay window over Segement

    U_SEG     = fft(u_seg)     / (Nest/2);          % We use actually a rect↙
windwow... so we need a upsacler
    U_SEG_WIN = fft(u_seg_win) / (sum(window)/2);   % Upscaler for window

    Pact = U_SEG .* conj(U_SEG); % two-sided power
    Pactw = U_SEG_WIN .* conj(U_SEG_WIN); % two-sided power

    Pseg = Pact(1:Nfreq);
    Psegw = Pactw(1:Nfreq);

    Pseg(1)   = Pseg(1)   / 4; % DC
    Psegw(1)   = Psegw(1)   / 4; % DC

    Pseg(end) = Pseg(end) / 4; % Nyquist (exists since Nest is even)
    Psegw(end) = Psegw(end) / 4; % Nyquist (exists since Nest is even)

    Pavg  = Pavg  + Pseg;       % Count spectrum up
    Pavgw = Pavgw + Psegw;
    Navg = Navg + 1;            % Upcounter to get mean
```

```
end

Pavg  = Pavg  / Navg;        % Mean of Spectrum
Pavgw = Pavgw / Navg;        % Mean of Spectrum

spectra = sqrt(Pavg);       % Go back from energie to amplitude
spectraw = sqrt(Pavgw);

figure(3)
subplot(2,1,1)
plot(freq, spectra); grid on
xlabel('Frequenz [Hz]'); ylabel('Amplitude');
title('Segmentiertes Spektrum ohne Fenster'); xlim([0 100])

subplot(2,1,2)
plot(freq, spectraw ); grid on
xlabel('Frequenz [Hz]'); ylabel('Amplitude');
title('Segmentiertes Spektrum mit Hann-Fenster'); xlim([0 100])

%% Get Spectrum with funciton

addpath ../lib/

[pxx, freq1] = estimate_spectra(u, window, Noverlap, Nest, Ts);
spectra_fun = sqrt(pxx); % power -> amplitude (dc needs to be scaled differently)

figure(5)
plot(freq1, spectra_fun); grid on;
set(gca, 'YScale')
title('Magnitude Spectra')
xlabel('Frequenz [Hz]'); ylabel('Amplitude')
xlim([0 100])
```

Projektcode